

Context-Free Languages

Regular Grammar

1. $01^* (0+1)^*$
2. $01(0+1)^*$
3. Letter followed by digit
4. Start and end with a
5. Generate all even integer upto 998
6. Alternate letters
7. a^{2n}
8. $(ab)^n$
9. $\{012, 120, 201\}$ generate these strings
10. $a^n b^m c^0$

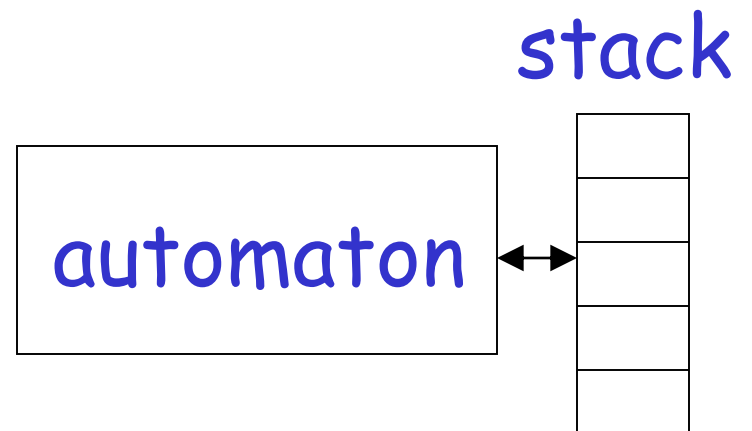
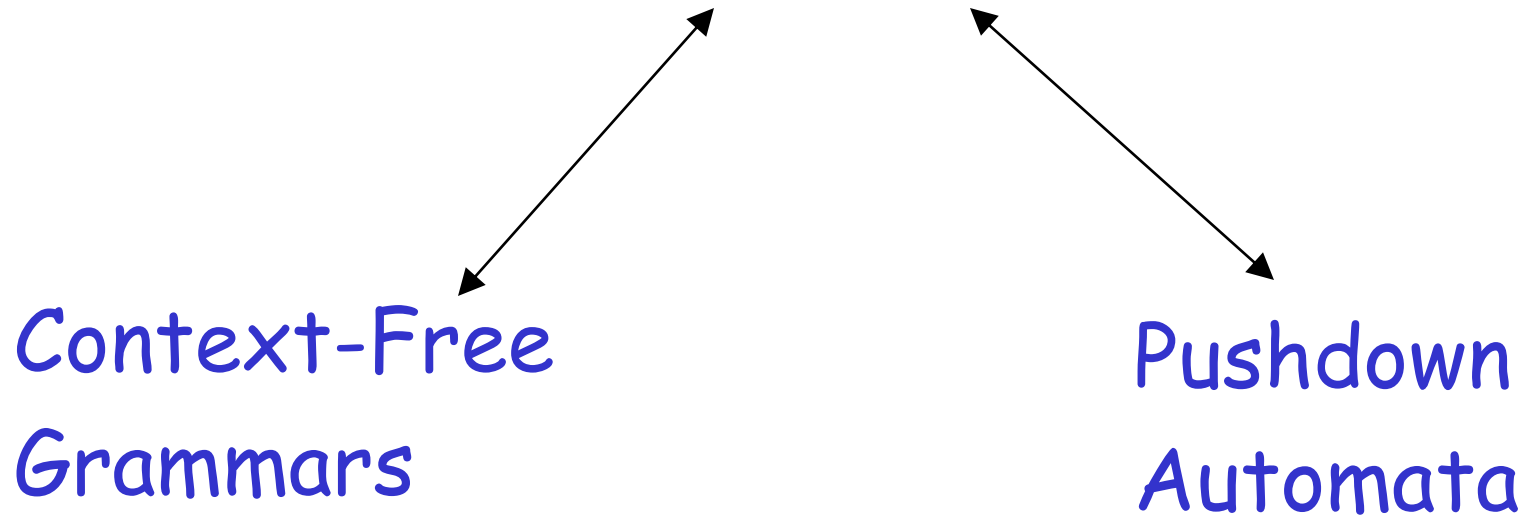
Context-Free Languages

$$\{a^n b^n\}$$

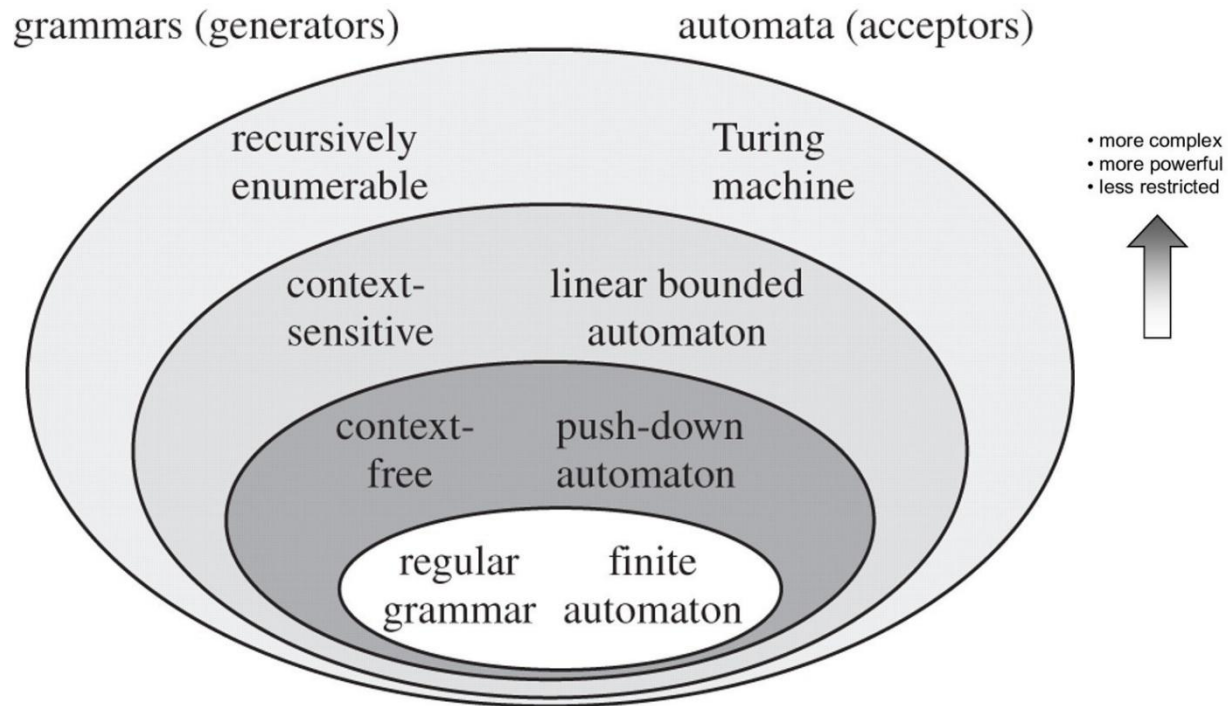
$$\{ww^R\}$$

Regular Languages

Context-Free Languages



Context-Free Grammars



Example

A context-free grammar G :

$$S \rightarrow aSb$$
$$S \rightarrow \lambda$$

One non terminal $\rightarrow$ terminal/non terminal

A derivation:

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$$

A context-free grammar G :

$$S \rightarrow aSb$$
$$S \rightarrow \lambda$$

Another derivation:

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaaSbbb \Rightarrow aaabbbb$$

$$S \rightarrow aSb$$

$$S \rightarrow \lambda$$

$$L(G) = \{a^n b^n : n \geq 0\}$$

Example

A context-free grammar G :

$$S \rightarrow aSa$$
$$S \rightarrow bSb$$
$$S \rightarrow \lambda$$

A derivation:

$$S \Rightarrow aSa \Rightarrow abSba \Rightarrow abba$$

A context-free grammar G :

$$S \rightarrow aSa$$
$$S \rightarrow bSb$$
$$S \rightarrow \lambda$$

Another derivation:

$$S \Rightarrow aSa \Rightarrow abSba \Rightarrow abaSaba \Rightarrow abaaba$$

$$S \rightarrow aSa$$

$$S \rightarrow bSb$$

$$S \rightarrow \lambda$$

$$L(G) =$$

$$S \rightarrow aSa$$

$$S \rightarrow bSb$$

$$S \rightarrow \lambda$$

$$L(G) = \{ww^R : w \in \{a,b\}^*\}$$

Example

A context-free grammar G :

$$S \rightarrow aSb$$

$$S \rightarrow SS$$

$$S \rightarrow \lambda$$

A derivation:

$$S \Rightarrow SS \Rightarrow aSbS \Rightarrow abS \Rightarrow ab$$

A context-free grammar G : $S \rightarrow aSb$

$$S \rightarrow SS$$

$$S \rightarrow \lambda$$

A derivation:

$$S \Rightarrow SS \Rightarrow aSbS \Rightarrow abS \Rightarrow abaSb \Rightarrow abab$$

$$S \rightarrow aSb$$

$$S \rightarrow SS$$

$$S \rightarrow \lambda$$

$$L(G) =$$

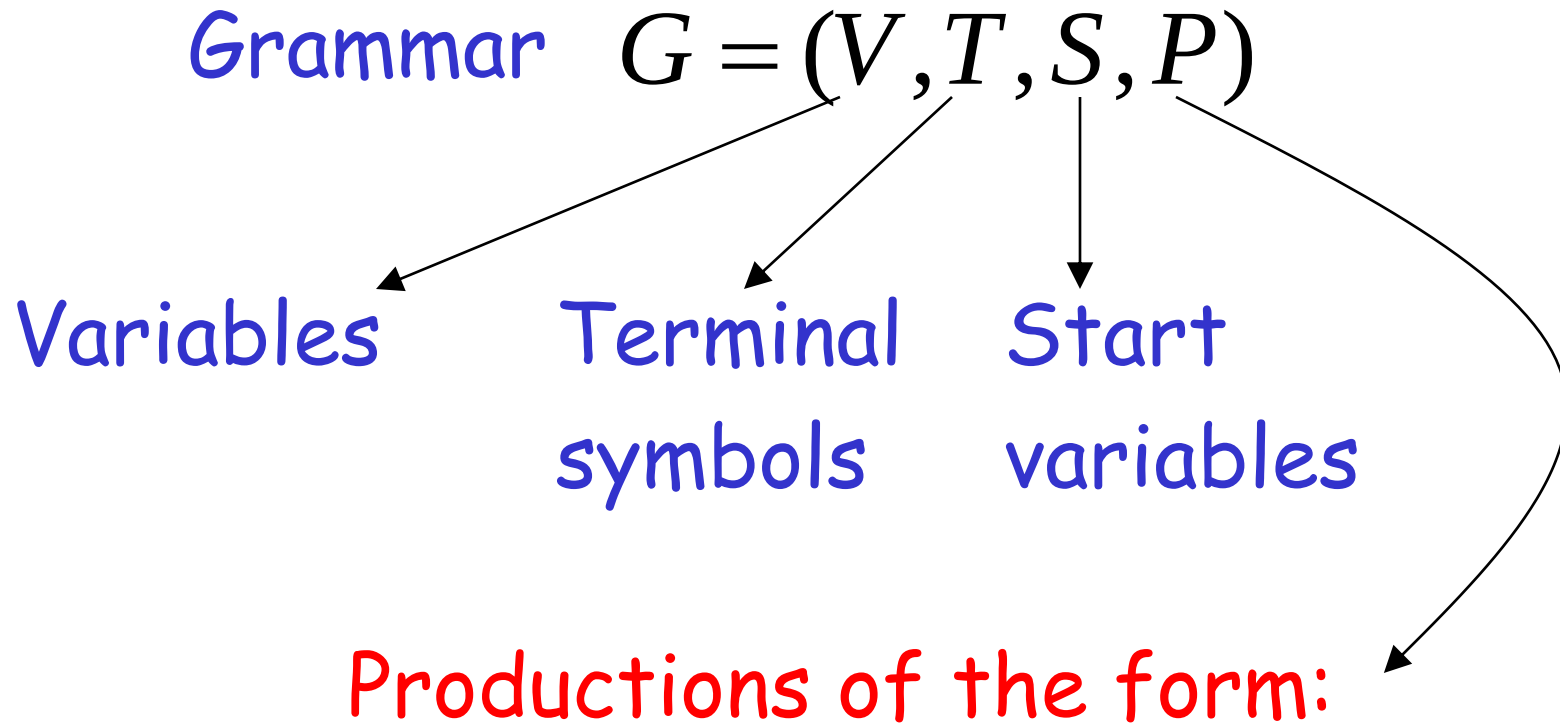
$$S \rightarrow aSb$$

$$S \rightarrow SS$$

$$S \rightarrow \lambda$$

$$L(G) = \{w : n_a(w) = n_b(w), \\ \text{and } n_a(v) \geq n_b(v) \\ \text{in any prefix } v\}$$

Definition: Context-Free Grammars



$$A \rightarrow \alpha$$

α is string of variables and terminals

Example of Context-Free Grammar

$$S \rightarrow aSb \mid \lambda$$

productions

$$P = \{S \rightarrow aSb, S \rightarrow \lambda\}$$

$$G = (V, T, S, P)$$

$V = \{S\}$
variables

$T = \{a, b\}$
terminals

start variable

Notation: Context-Free Grammars

- Lower case letters near beginning of alphabet (e.g. $a, b, c, \dots$) are terminal symbols
- Upper case letters near beginning of alphabet (e.g. $A, B, C, \dots$) are variables
- Lower case letters near end of alphabet (e.g. $w, x, \dots$) are strings of terminals
- Upper case letters near end of alphabet (e.g. $X, Y, \dots$) are either terminals or strings
- Lower case Greek letters (e.g. $\alpha, \beta, \gamma, \dots$) are strings of terminals and/or variables

Definition: Context-Free Languages

A language L is context-free

if and only if

there is a grammar G with $L = L(G)$

Recall

Language of a Grammar

For a grammar G
with start variable S :

$$L(G) = \{w : S \Rightarrow w, \quad w \in T^*\}$$

String of terminals

Example:

context-free grammar $G : \boxed{S \rightarrow aSb \mid \lambda}$

$$L(G) = \{a^n b^n : n \geq 0\}$$

Since, there is derivation

$$S \xRightarrow{*} a^n b^n \quad \text{for any } n \geq 0$$

Example:

$$L = \{a^n b^n : n \geq 0\}$$

is a context-free language

since context-free grammar G :

$$S \rightarrow aSb \mid \lambda$$

generates $L(G) = L$

Examples

$a^n b a^n, \text{ if } n \geq 0$

$ww, w \in \{a, b\}^*$

$S \rightarrow aSa | b$

This CSL, no CFG or PDA

$a^n b a^n, \text{ if } n \geq 1$

$a^n b^m c^{n+m}$

$S \rightarrow aSa | aba$

$S \rightarrow \lambda | aSc | B$
 $B \rightarrow bBc | \lambda$

$$a^n b^n c^m$$

$$a^{2n} b^{3n}$$

$$S \rightarrow aaSbbb \mid \lambda$$

$$S \rightarrow AB$$

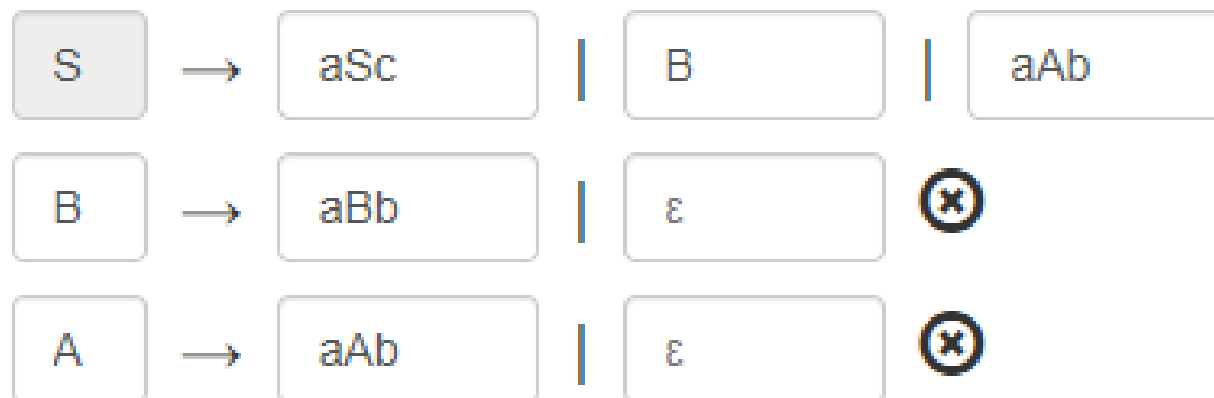
$$A \rightarrow aAb \mid \lambda$$

$$B \rightarrow cB \mid \lambda$$

$$a^n b^{2n}$$

$$S \rightarrow aSbb \mid \lambda$$

$$a^{n+m}b^nc^m$$



Verify

This is the CFG you have input above:

Start symbol: **S**

S $\rightarrow$ aSc | **B** | aAb

B $\rightarrow$ aBb | ϵ

A $\rightarrow$ aAb | ϵ

Some strings from the language of this grammar:

```
aaaabccc  
aaaaaaaaabbbbbccc  
aaaaaaaaabbbbcccc  
aaaaaaabbbcccc  
aaaaaaabbbbbbbc  
aaaaabbbbb  
aaaaaaabbcccccc  
 $\epsilon$   
ac  
aaaaaaabbcccccc
```

$$n_a(w) \bmod 3 = 0$$

→ | |

+ Click here or press "Enter" for a new production

Verify

This is the CFG you have input above:

Start symbol: **S**

S → **SaSaSaS** | **bS** | **ε**

Some strings from the language of this grammar:

```

aaabaaabbaaaabaa
bb
bbbaaaaababbbaa
aaabb
baaaabaa
abbaabaabaabaa
aabba
ε
bababa
aaaaaa
  
```

Test

$a^n b^m: (n + m) \text{ is even}$

S	→	Aaa		Bbb		aaB		aAbB		AB		ε
A	→	Aaa		ε	⊗							
B	→	Bbb		ε	⊗							

+ Click here or press "Enter" for a new production

Verify

This is the CFG you have input above:

Start symbol: **S**

S → **A**aa | **B**bb | aa**B** | aA**bB** | **AB** | ε

A → **A**aa | ε

B → **B**bb | ε

Some strings from the language of this grammar:

```
abbbbbbbbbbbbbbb
abbbbbbbbbbb
ε
bbbbbbbbbbbbbbbb
aaaabbbbbbbbbbb
aabbbbbbbbbbb
aaaa
aaaaaabbbb
aaaaaabbbbbbb
```

Simplified Version

Start symbol: **S**

S $\rightarrow$ **AB** | **AaBb** | ϵ

A $\rightarrow$ **Aaa** | ϵ

B $\rightarrow$ **Bbb** | ϵ

Some strings from the language of this grammar:

aaaaaab

aabbbbbbbbbbbb

aaaaaa

aaaabbbbbbb

ϵ

aaaabbbbbbbb

bbbbbbbb

aaaabb

bbbb

abbbbbbbbbbb

atleast two occurences of consecutive zeros

→

→ | |

Click here or press "Enter" for a new production

Verify

This is the CFG you have input above:

Start symbol: **S**

S → **A00A00A**

A → **0A** | **1A** | **ε**

Some strings from the language of this grammar:

```
000000
011000010000100
0000
0100000
0000010100101000
00001
0111000100100
0011100100
000000010111011
0000011
```


$$(a + b)^*$$

→ | |

[+ Click here or press "Enter" for a new production](#)

Verify

This is the CFG you have input above:

Start symbol: **S**

S → a**S** | b**S** | ε

Some strings from the language of this grammar:

```
ababaabbbaaa  
aabaabbbaab  
a  
bab  
bbbbaaaaaabaab  
bbbbaa  
babaaba  
abaa  
abaabbbaba  
aab
```

Derivation Order and Derivation Trees

Derivation Order

Consider the following example grammar with 5 productions:

- | | | |
|-----------------------|----------------------------|----------------------------|
| 1. $S \rightarrow AB$ | 2. $A \rightarrow aaA$ | 4. $B \rightarrow Bb$ |
| | 3. $A \rightarrow \lambda$ | 5. $B \rightarrow \lambda$ |

Derive a string aab

- | | | |
|-----------------------|----------------------------|----------------------------|
| 1. $S \rightarrow AB$ | 2. $A \rightarrow aaA$ | 4. $B \rightarrow Bb$ |
| | 3. $A \rightarrow \lambda$ | 5. $B \rightarrow \lambda$ |

Leftmost derivation order of string aab :

$$\begin{array}{ccccccccc} & 1 & & 2 & & 3 & & 4 & & 5 \\ S & \Rightarrow & AB & \Rightarrow & aaAB & \Rightarrow & aaB & \Rightarrow & aaBb & \Rightarrow & aab \end{array}$$

At each step, we substitute the
leftmost variable

- | | | |
|-----------------------|----------------------------|----------------------------|
| 1. $S \rightarrow AB$ | 2. $A \rightarrow aaA$ | 4. $B \rightarrow Bb$ |
| | 3. $A \rightarrow \lambda$ | 5. $B \rightarrow \lambda$ |

Rightmost derivation order of string aab :

$$\begin{array}{ccccccccc} & 1 & & 4 & & 5 & & 2 & & 3 \\ S & \Rightarrow & AB & \Rightarrow & ABb & \Rightarrow & Ab & \Rightarrow & aaAb & \Rightarrow & aab \end{array}$$

At each step, we substitute the
rightmost variable

- | | | |
|-----------------------|----------------------------|----------------------------|
| 1. $S \rightarrow AB$ | 2. $A \rightarrow aaA$ | 4. $B \rightarrow Bb$ |
| | 3. $A \rightarrow \lambda$ | 5. $B \rightarrow \lambda$ |

Leftmost derivation of aab :

$$\begin{array}{ccccccccc}
 1 & & 2 & & 3 & & 4 & & 5 \\
 S & \Rightarrow & AB & \Rightarrow & aaAB & \Rightarrow & aaB & \Rightarrow & aaBb & \Rightarrow & aab
 \end{array}$$

Rightmost derivation of aab :

$$\begin{array}{ccccccccc}
 1 & & 4 & & 5 & & 2 & & 3 \\
 S & \Rightarrow & AB & \Rightarrow & ABb & \Rightarrow & Ab & \Rightarrow & aaAb & \Rightarrow & aab
 \end{array}$$

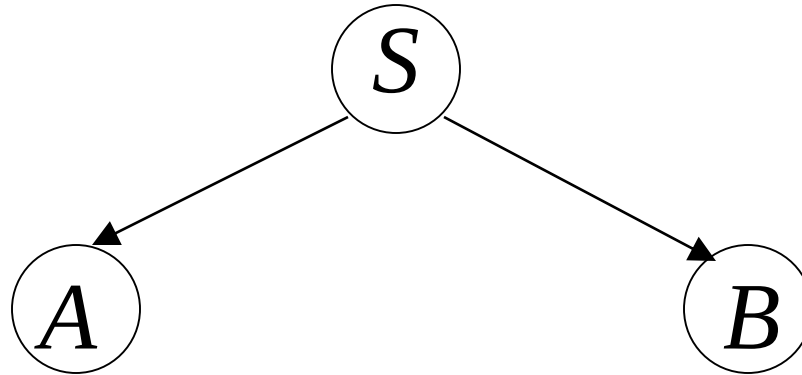
Derivation Trees

$$S \rightarrow AB$$

$$A \rightarrow aaA \mid \lambda$$

$$B \rightarrow Bb \mid \lambda$$

$$S \Rightarrow AB$$

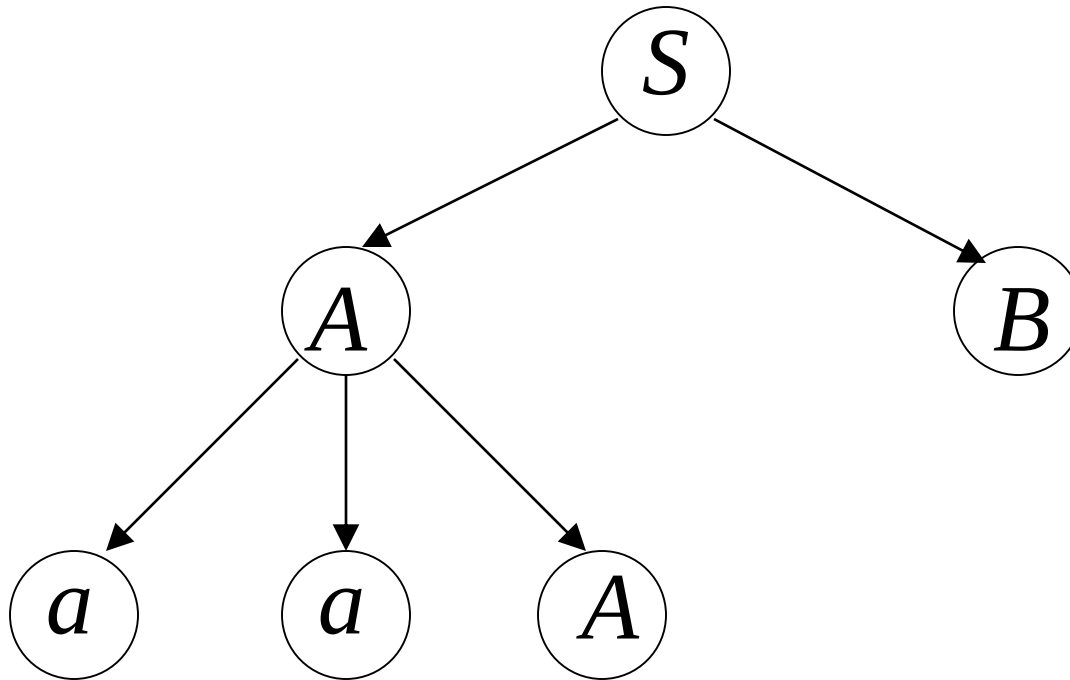


$$S \rightarrow AB$$

$$A \rightarrow aaA \mid \lambda$$

$$B \rightarrow Bb \mid \lambda$$

$$S \Rightarrow AB \Rightarrow aaAB$$

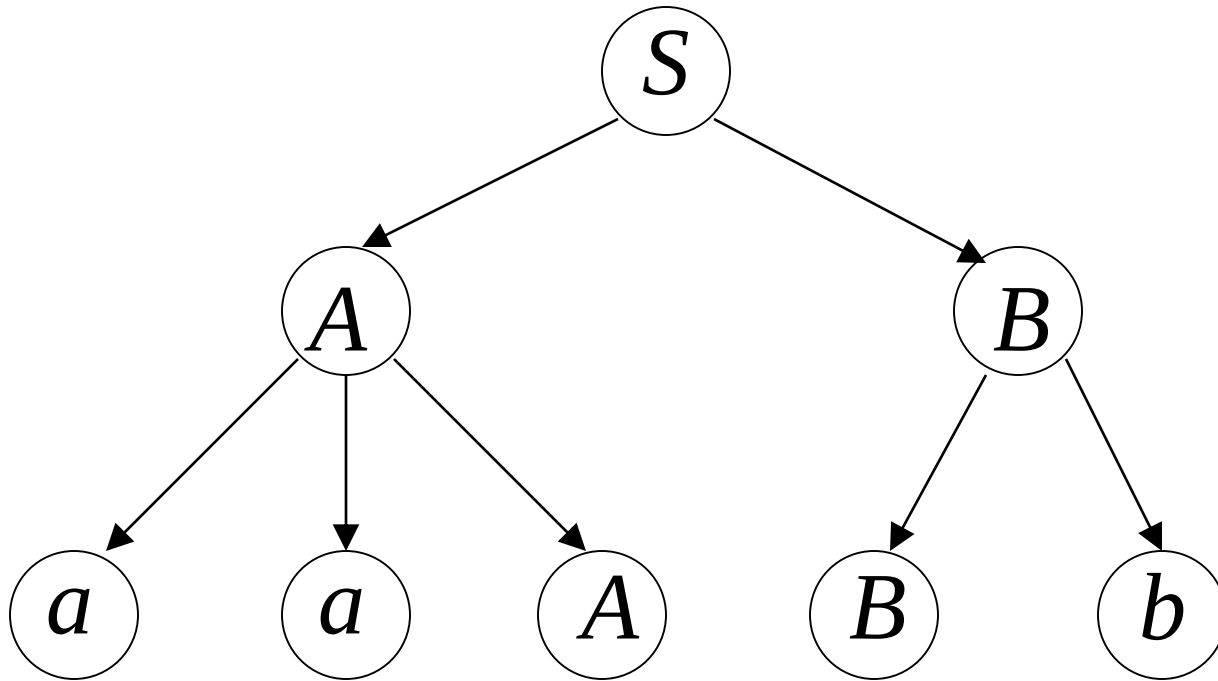


$$S \rightarrow AB$$

$$A \rightarrow aaA \mid \lambda$$

$$B \rightarrow Bb \mid \lambda$$

$$S \Rightarrow AB \Rightarrow aaAB \Rightarrow aaABb$$

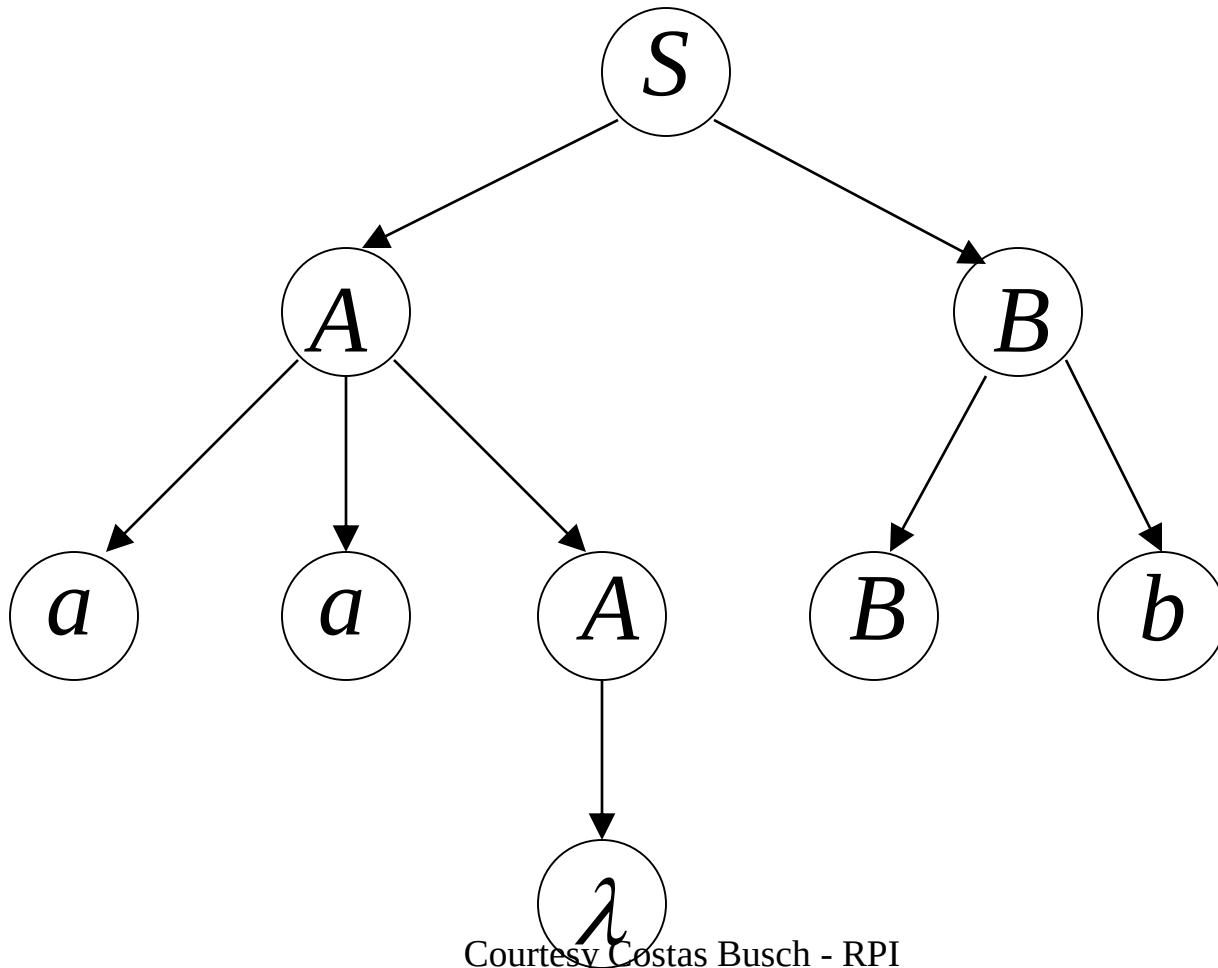


$$S \rightarrow AB$$

$$A \rightarrow aaA \mid \lambda$$

$$B \rightarrow Bb \mid \lambda$$

$$S \Rightarrow AB \Rightarrow aaAB \Rightarrow aaABb \Rightarrow aaBb$$



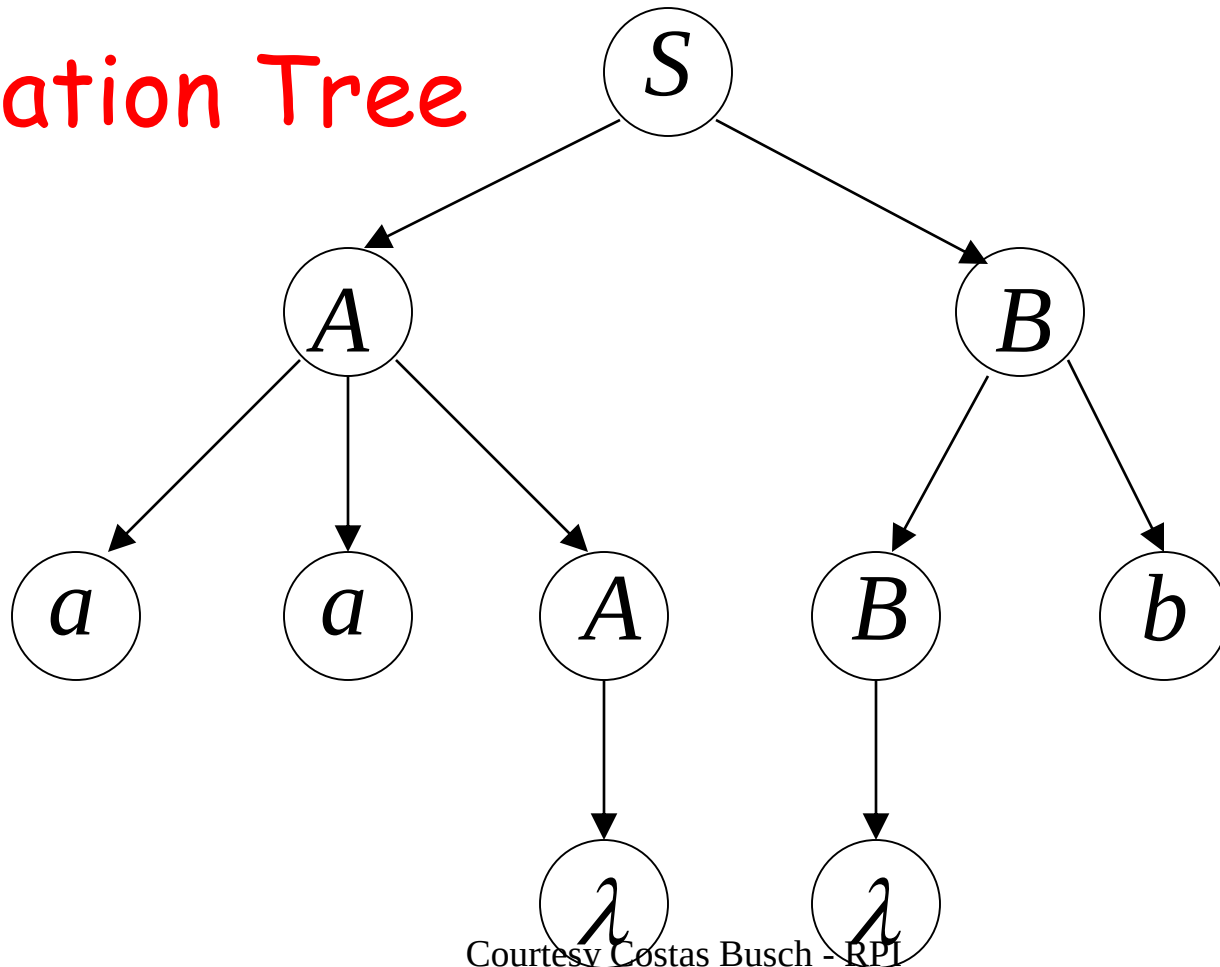
$$S \rightarrow AB$$

$$A \rightarrow aaA \mid \lambda$$

$$B \rightarrow Bb \mid \lambda$$

$$S \Rightarrow AB \Rightarrow aaAB \Rightarrow aaABb \Rightarrow aaBb \Rightarrow aab$$

Derivation Tree



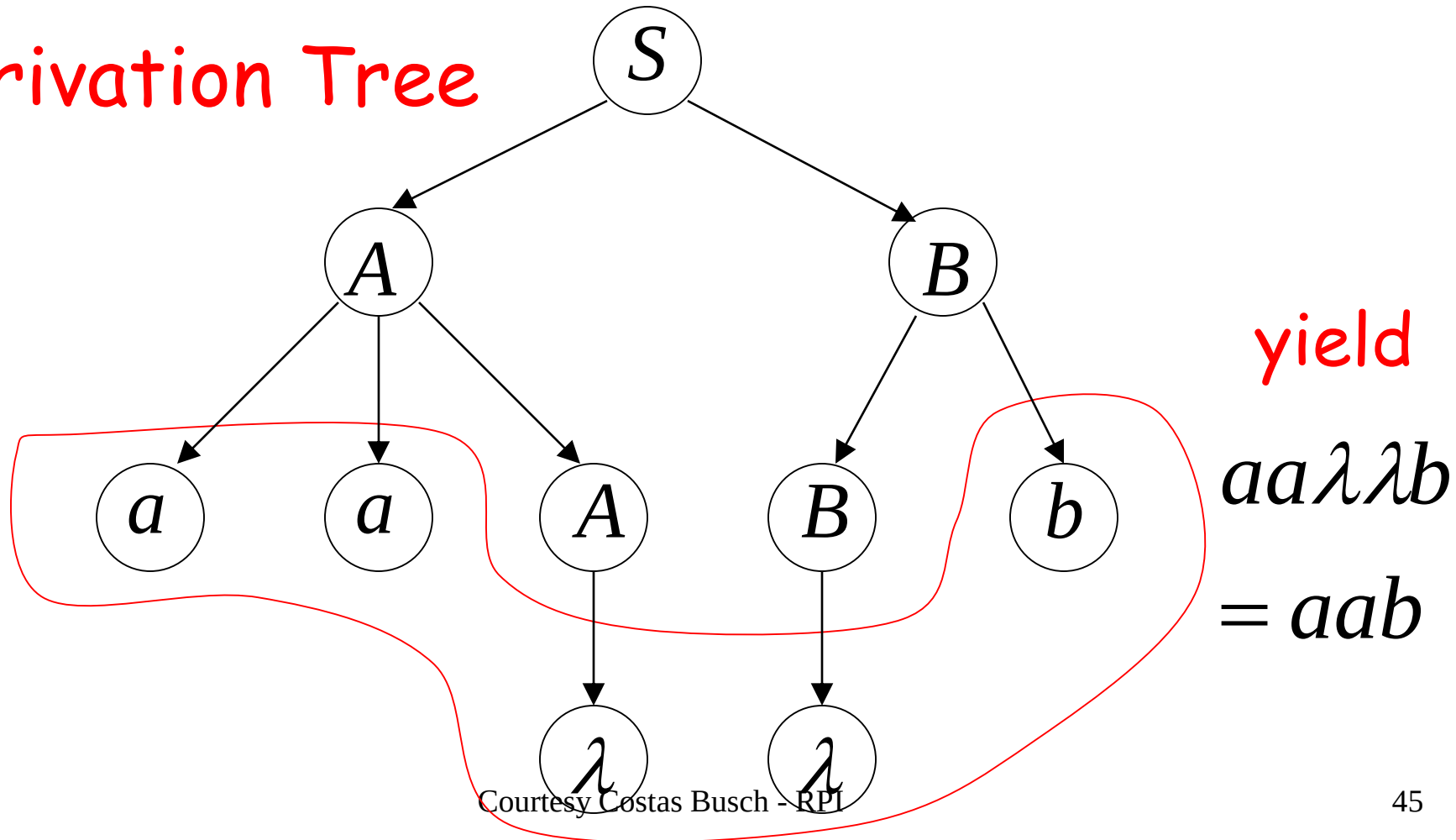
$$S \rightarrow AB$$

$$A \rightarrow aaA \mid \lambda$$

$$B \rightarrow Bb \mid \lambda$$

$$S \Rightarrow AB \Rightarrow aaAB \Rightarrow aaABb \Rightarrow aaBb \Rightarrow aab$$

Derivation Tree



Partial Derivation Trees

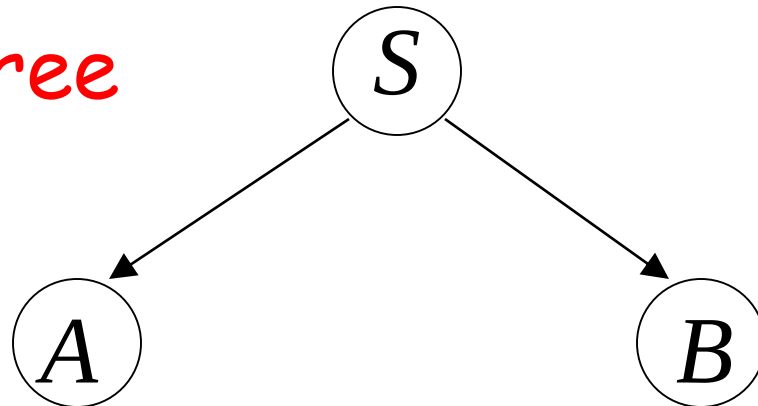
$$S \rightarrow AB$$

$$A \rightarrow aaA \mid \lambda$$

$$B \rightarrow Bb \mid \lambda$$

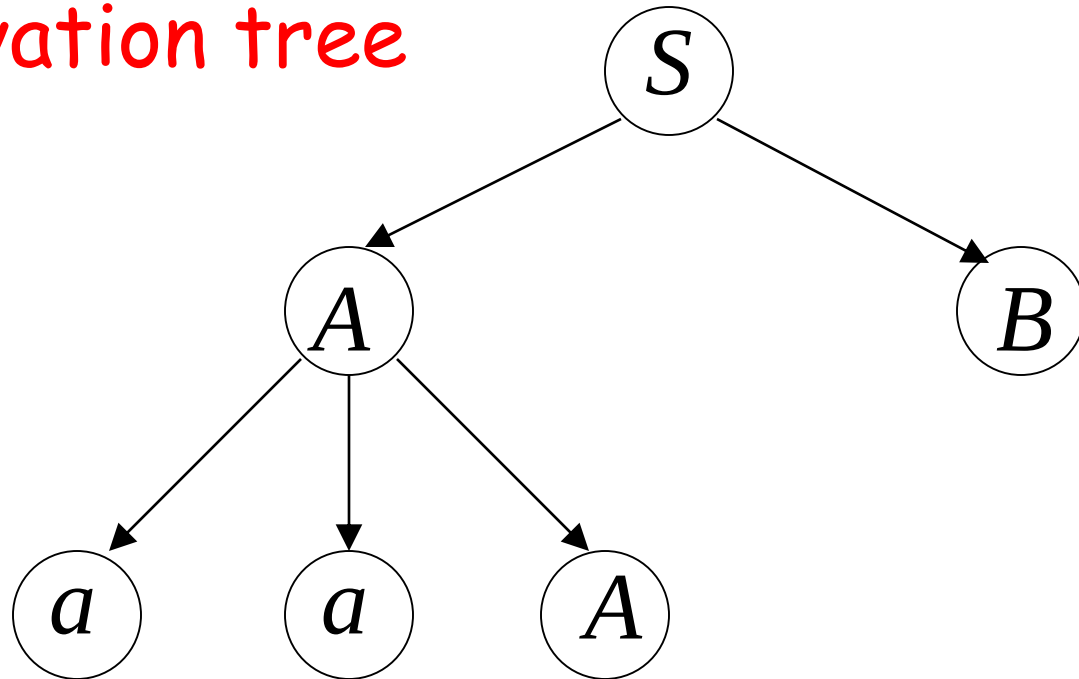
$$S \Rightarrow AB$$

Partial derivation tree



$$S \Rightarrow AB \Rightarrow aaAB$$

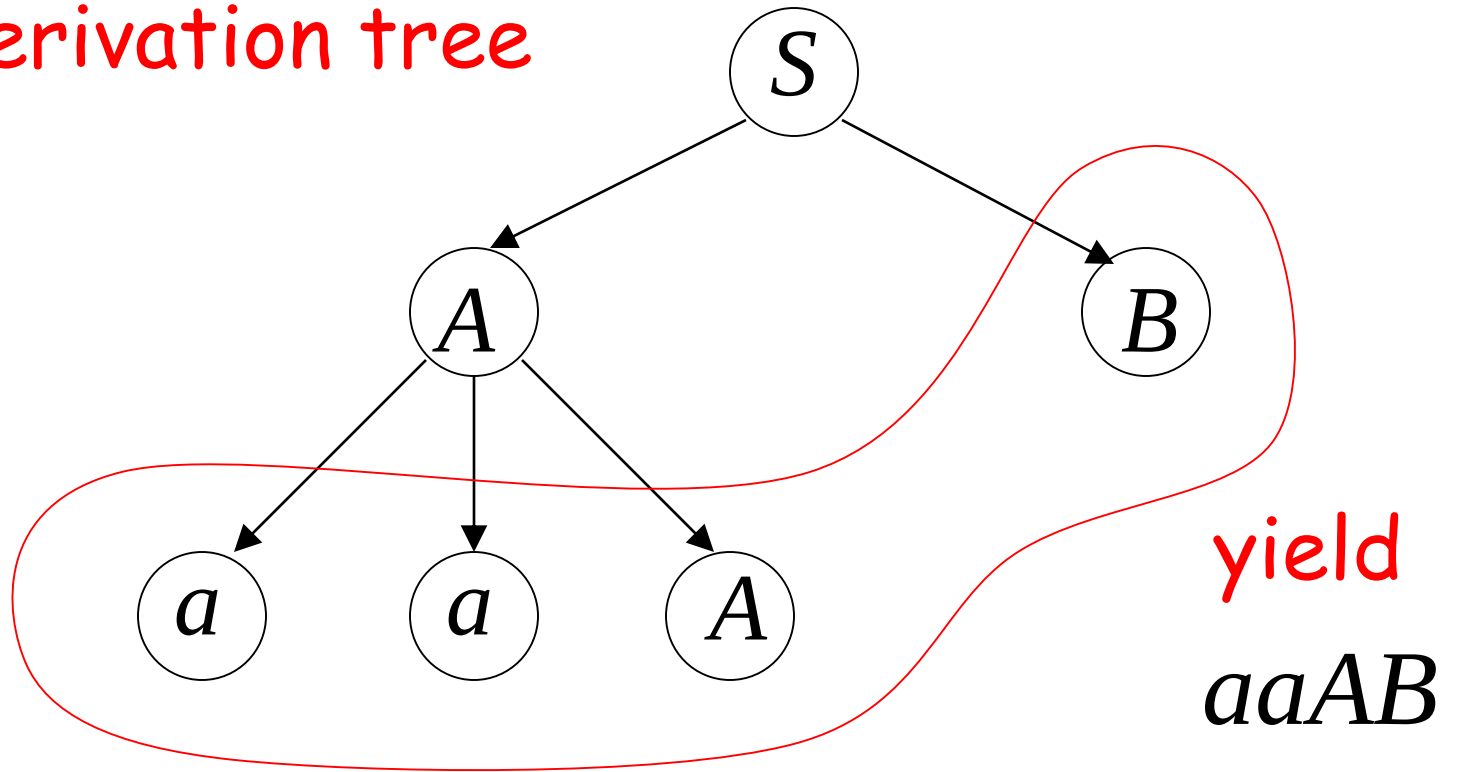
Partial derivation tree



$$S \Rightarrow AB \Rightarrow aaAB$$

sentential
form

Partial derivation tree



Sometimes, derivation order doesn't matter

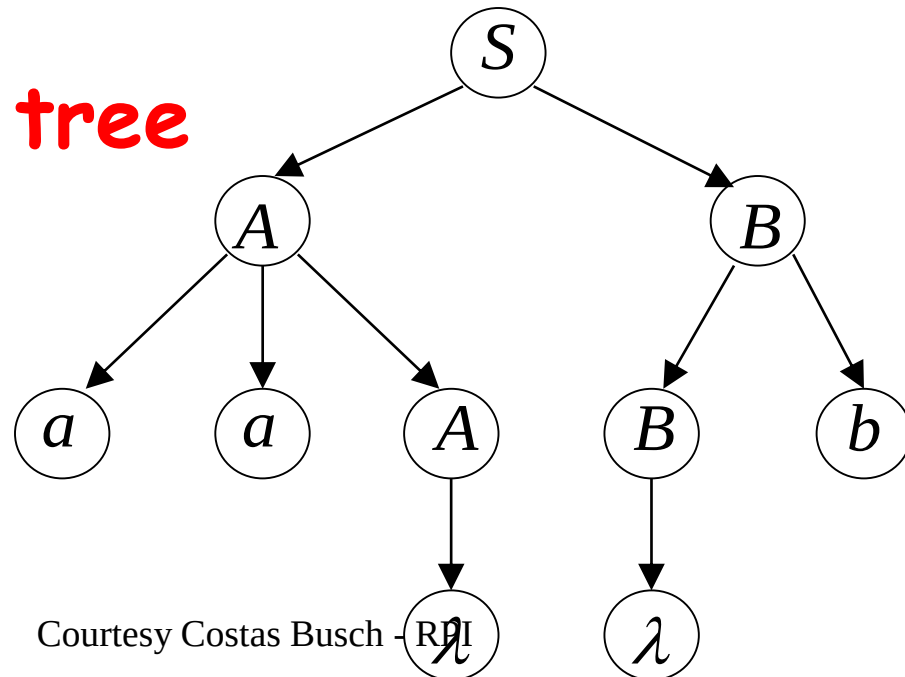
Leftmost:

$S \Rightarrow AB \Rightarrow aaAB \Rightarrow aaB \Rightarrow aaBb \Rightarrow aab$

Rightmost:

$S \Rightarrow AB \Rightarrow ABb \Rightarrow Ab \Rightarrow aaAb \Rightarrow aab$

Same derivation tree



Ambiguity

A CFG for C++ compound statements:

```
<compound stmt> --> { <stmt list> }  
<stmt list> --> <stmt> <stmt list> | epsilon  
<stmt> --> <compound stmt>  
<stmt> --> if ( <expr> ) <stmt>  
<stmt> --> if ( <expr> ) <stmt> else <stmt>  
<stmt> --> while ( <expr> ) <stmt>  
<stmt> --> do <stmt> while ( <expr> ) ;  
<stmt> --> for ( <stmt> <expr> ; <expr> ) <stmt>  
<stmt> --> case <expr> : <stmt>  
<stmt> --> switch ( <expr> ) <stmt>  
<stmt> --> break ; | continue ;  
<stmt> --> return <expr> ; | goto <id> ;
```

Grammar for mathematical expressions

$$E \rightarrow E + E \mid E * E \mid (E) \mid a$$

Example strings:

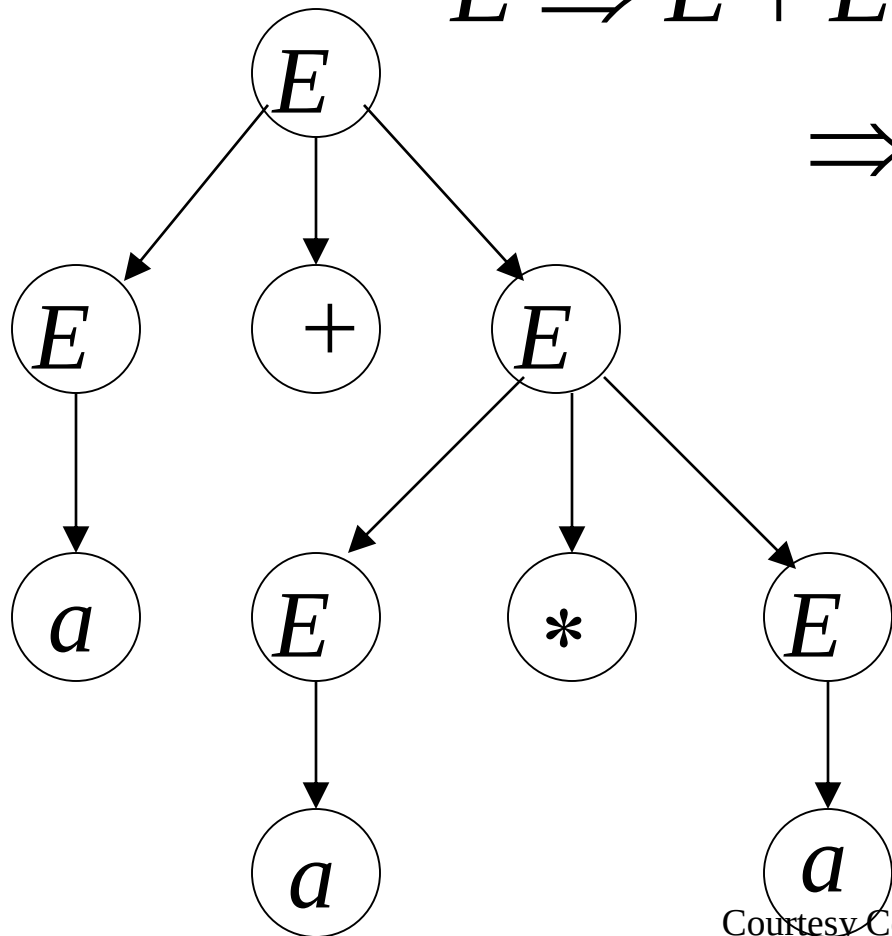
$$(a + a) * a + (a + a * (a + a))$$



Denotes any number

$$E \rightarrow E + E \mid E * E \mid (E) \mid a$$

$$\begin{aligned} E &\Rightarrow E + E \Rightarrow a + E \Rightarrow a + E * E \\ &\Rightarrow a + a * E \Rightarrow a + a * a \end{aligned}$$

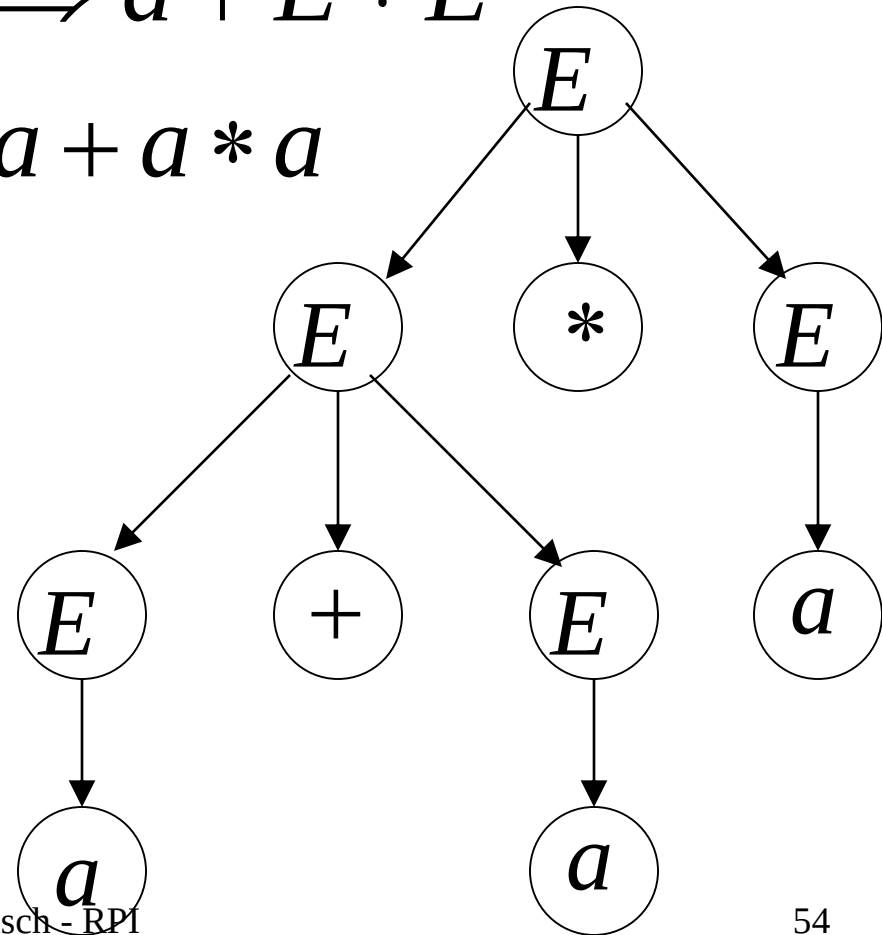


A leftmost derivation
for $a + a * a$

$$E \rightarrow E + E \mid E * E \mid (E) \mid a$$

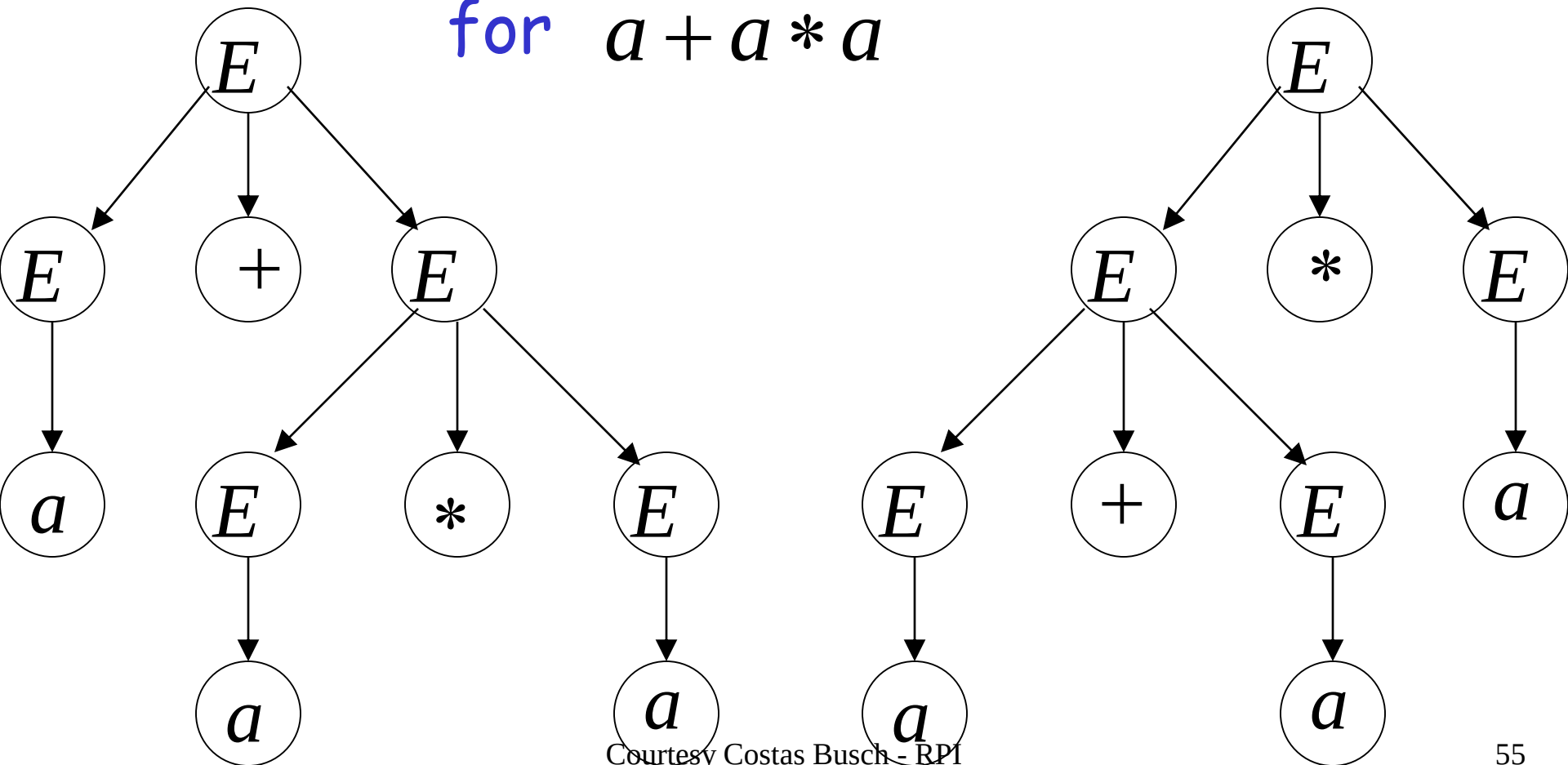
$$\begin{aligned} E &\Rightarrow E * E \Rightarrow E + E * E \Rightarrow a + E * E \\ &\Rightarrow a + a * E \Rightarrow a + a * a \end{aligned}$$

Another
leftmost derivation
for $a + a * a$



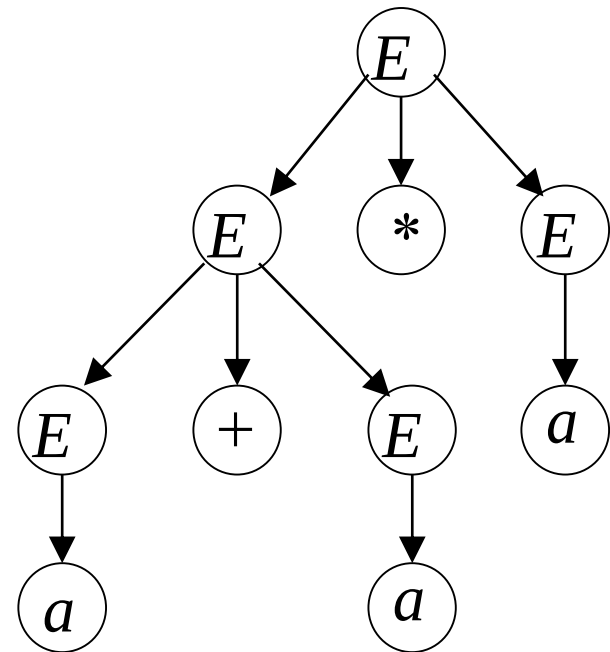
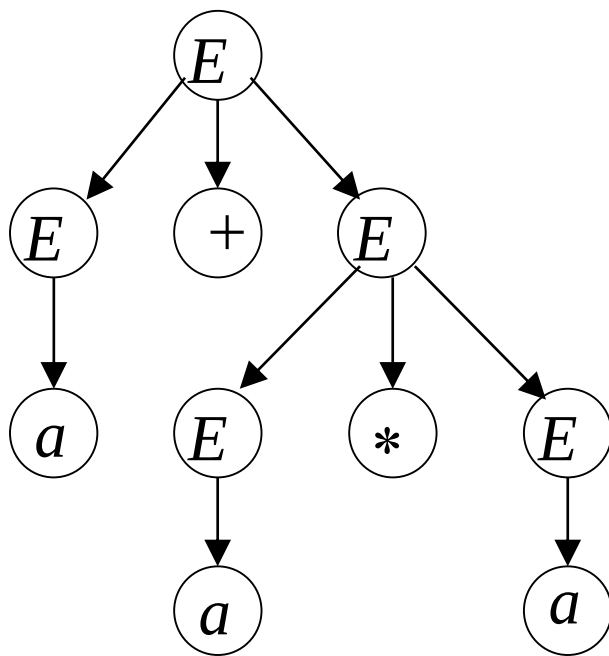
$$E \rightarrow E + E \mid E * E \mid (E) \mid a$$

Two derivation trees
for $a + a * a$



The grammar $E \rightarrow E + E \mid E * E \mid (E) \mid a$
is ambiguous:

string $a + a * a$ has two derivation trees



The grammar $E \rightarrow E + E \mid E * E \mid (E) \mid a$
is ambiguous:

string $a + a * a$ has two leftmost derivations

$$\begin{aligned} E &\Rightarrow E + E \Rightarrow a + E \Rightarrow a + E * E \\ &\Rightarrow a + a * E \Rightarrow a + a * a \end{aligned}$$

$$\begin{aligned} E &\Rightarrow E * E \Rightarrow E + E * E \Rightarrow a + E * E \\ &\Rightarrow a + a * E \Rightarrow a + a * a \end{aligned}$$

Definition:

A context-free grammar G is **ambiguous**

if some string $w \in L(G)$ has:

two or more derivation trees

In other words:

A context-free grammar G is **ambiguous**

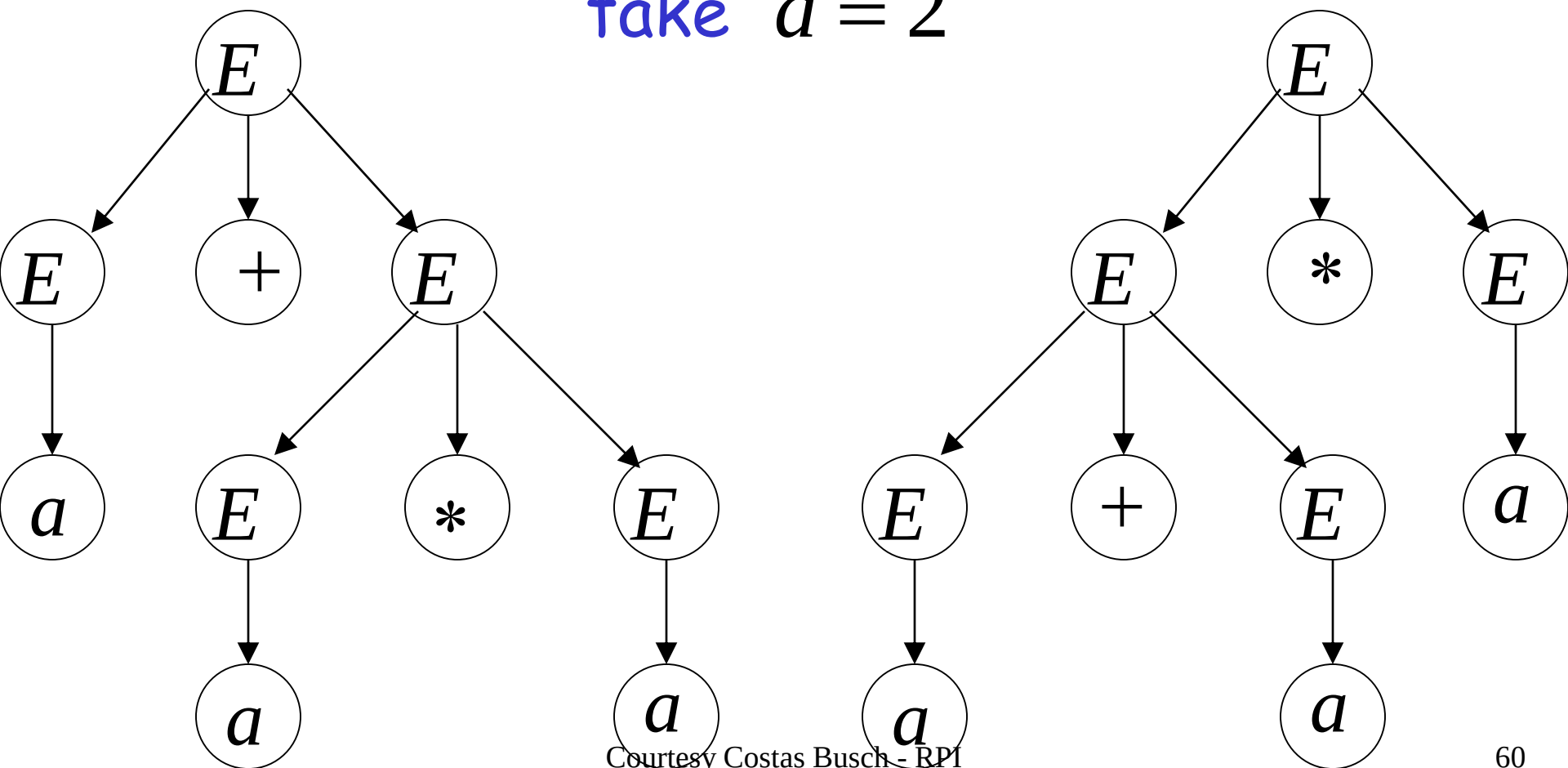
if some string $w \in L(G)$ has:

two or more leftmost derivations
(or rightmost)

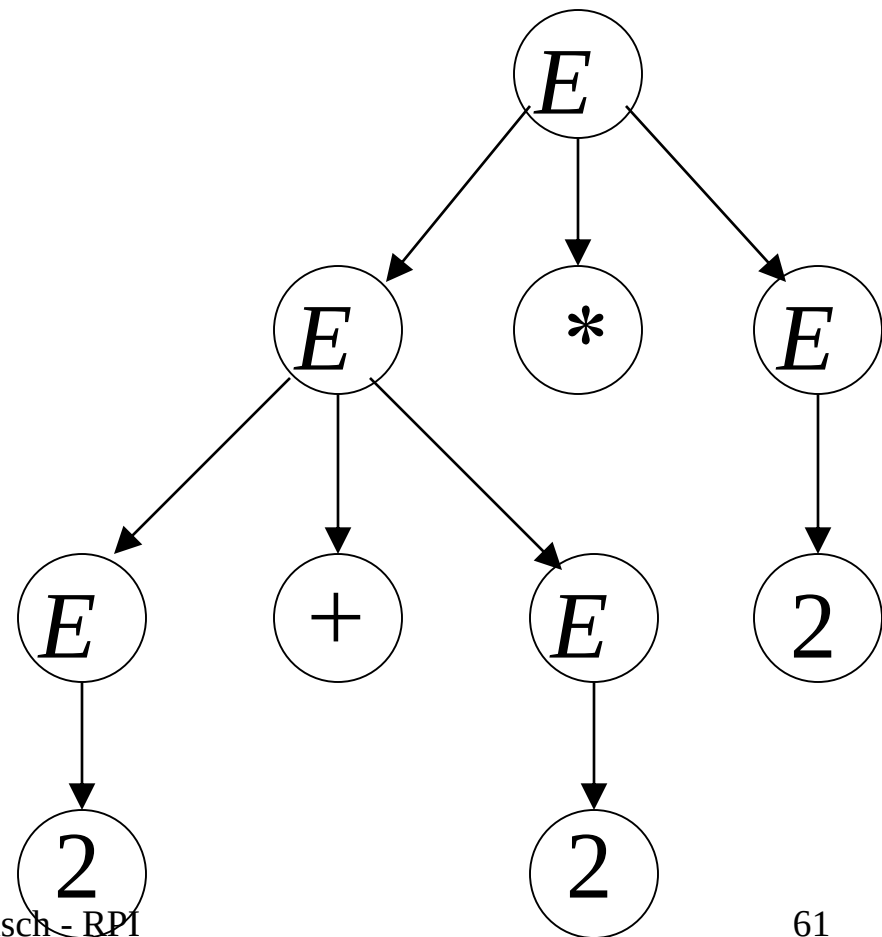
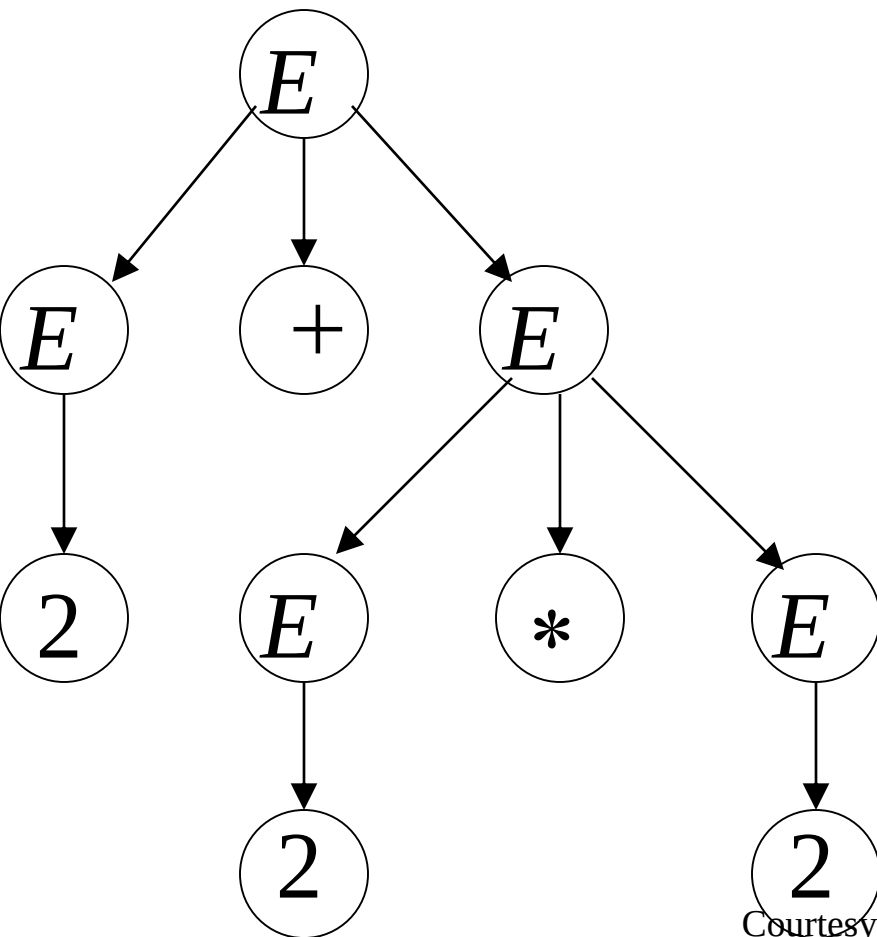
Why do we care about ambiguity?

$$a + a * a$$

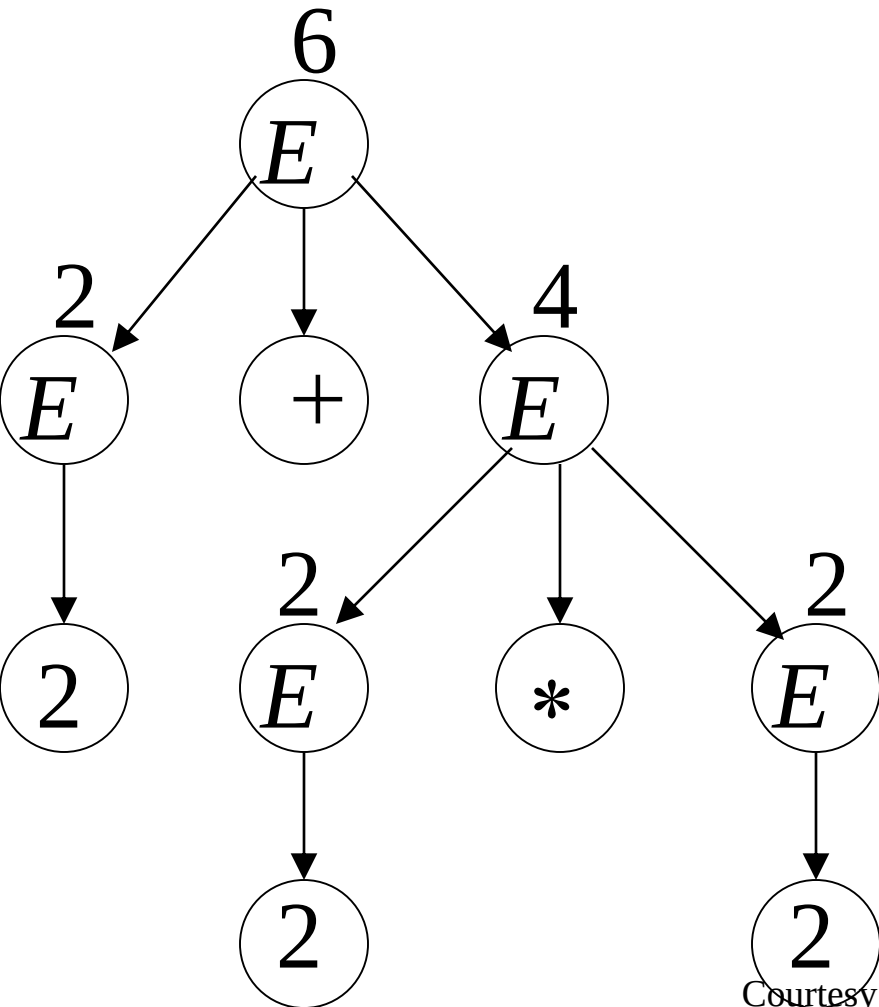
take $a = 2$



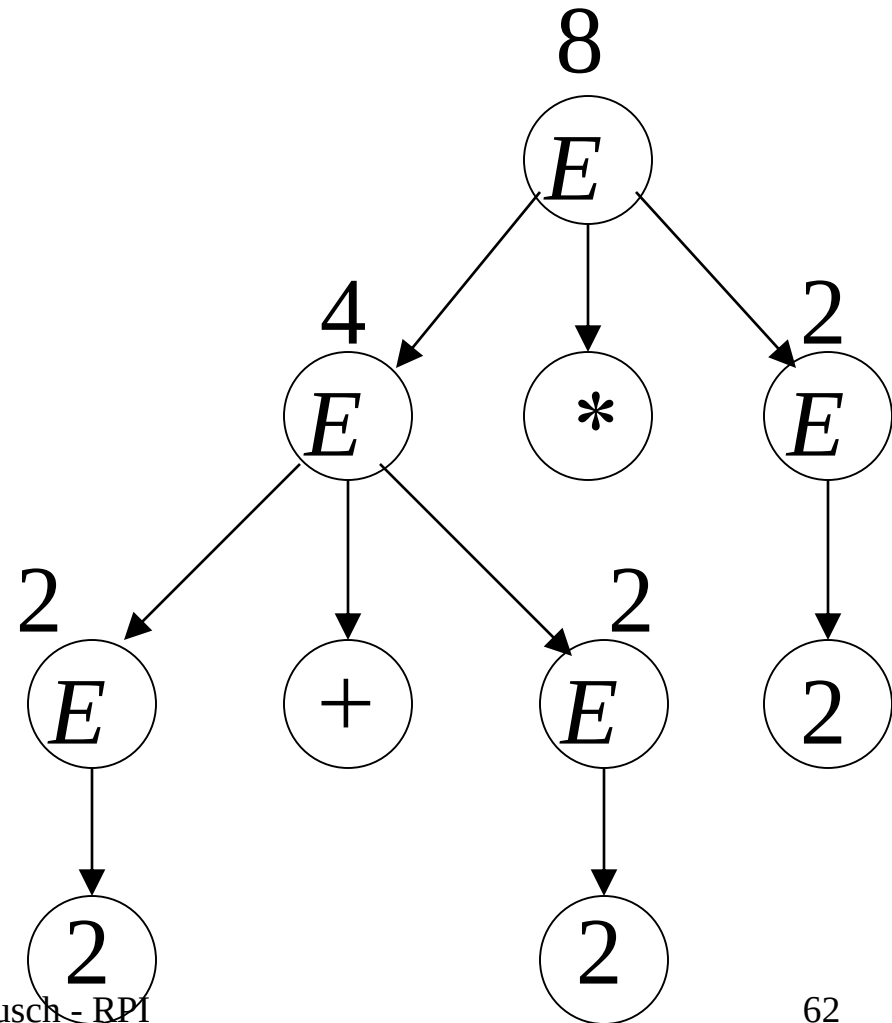
$$2 + 2 * 2$$



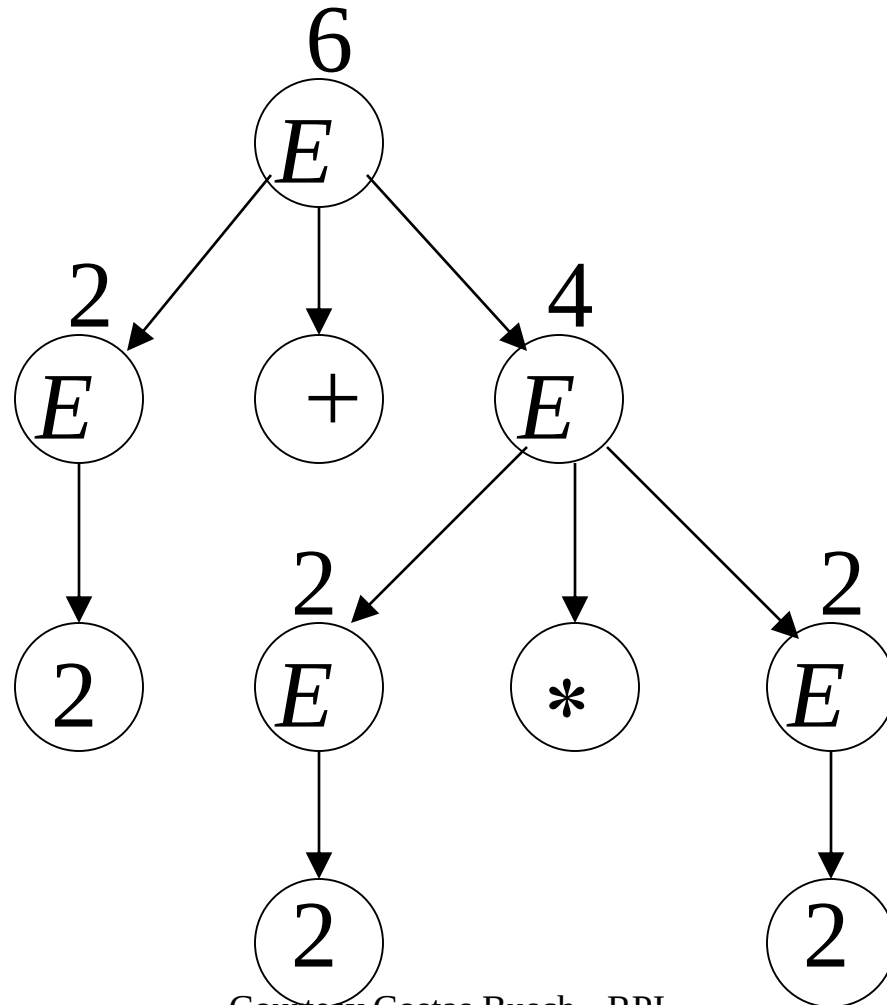
$$2 + 2 * 2 = 6$$



$$2 + 2 * 2 = 8$$



Correct result: $2 + 2 * 2 = 6$



Courtesy Costas Busch - RPI

- Ambiguity is **bad** for programming languages
- We want to remove ambiguity

Ambiguous Grammar:

A context-free grammar G is ambiguous if there is a string $w \in L(G)$ which has:

two different derivation trees

or

two leftmost derivations

(Two different derivation trees give two different leftmost derivations and vice-versa)

In general, ambiguity is bad
and we want to remove it

Sometimes it is possible to find
a non-ambiguous grammar for a language

But, in general it is difficult to achieve this

A successful example:

Ambiguous Grammar

$$\begin{aligned} E &\rightarrow E + E \\ E &\rightarrow E * E \\ E &\rightarrow (E) \\ E &\rightarrow a \end{aligned}$$

Equivalent

Non-Ambiguous Grammar

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid a \end{aligned}$$

generates the same language

$$\begin{aligned}
 E &\Rightarrow E + T \Rightarrow T + T \Rightarrow F + T \Rightarrow a + T \Rightarrow a + T * F \\
 &\Rightarrow a + F * F \Rightarrow a + a * F \Rightarrow a + a * a
 \end{aligned}$$

$$E \rightarrow E + T$$

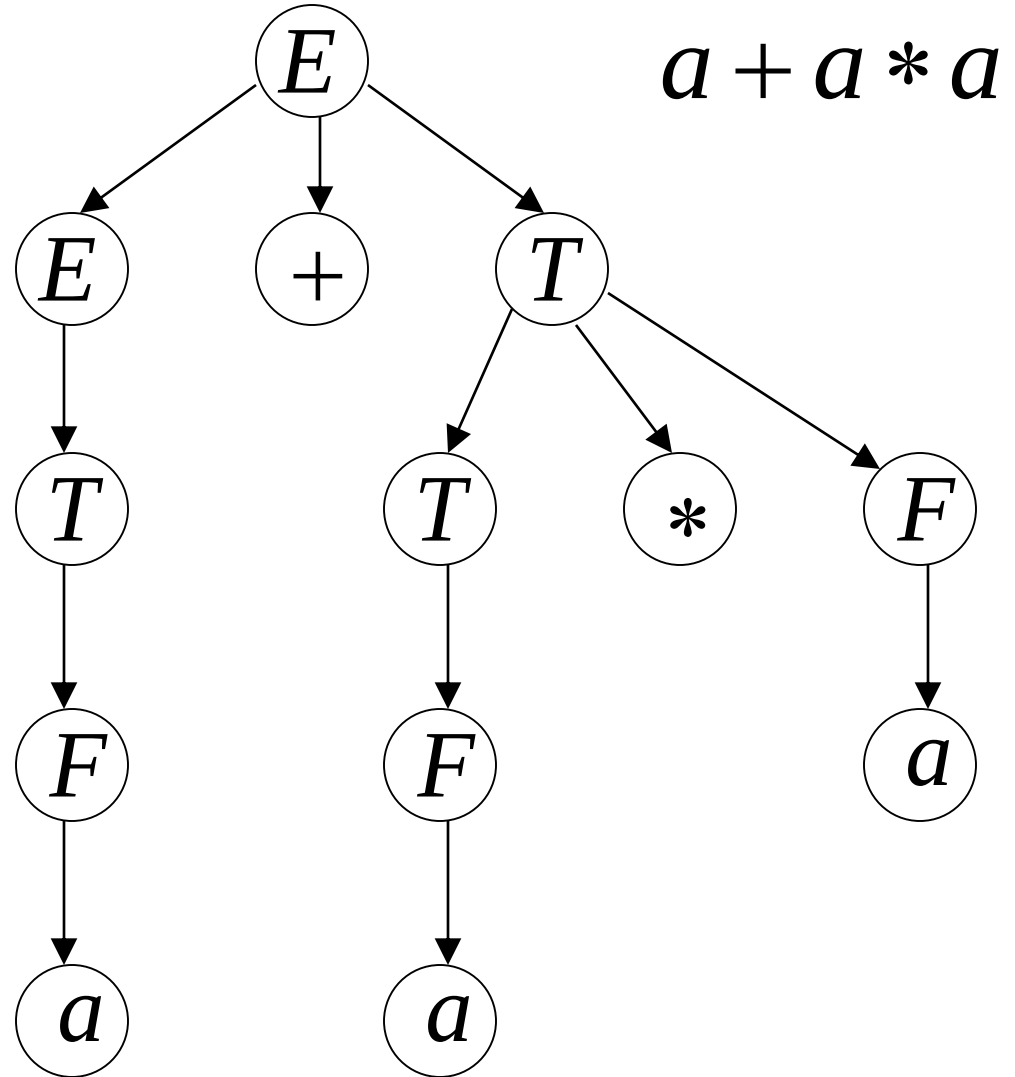
$$E \rightarrow T$$

$$T \rightarrow T * F$$

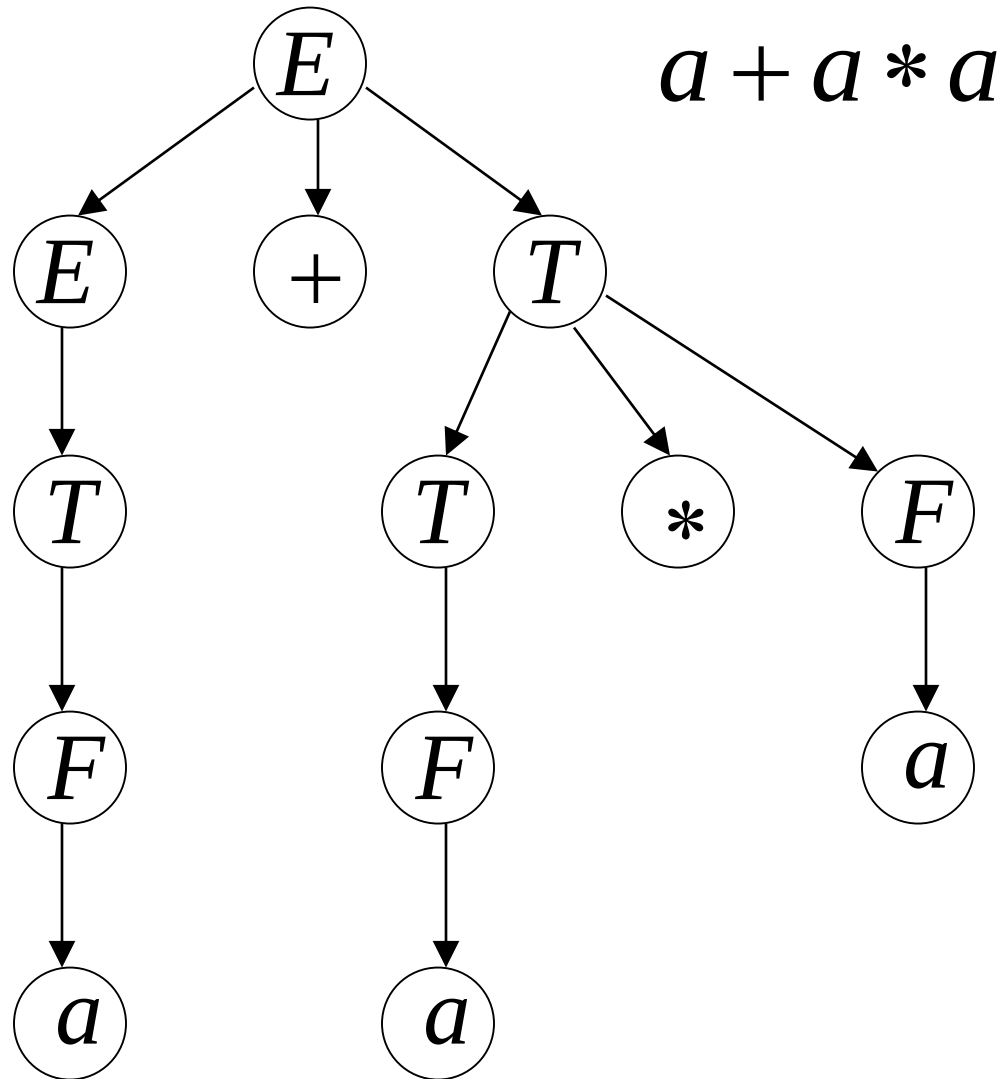
$$T \rightarrow F$$

$$F \rightarrow (E)$$

$$F \rightarrow a$$



Unique derivation tree



The grammar G :

$$E \rightarrow E + T$$
$$E \rightarrow T$$
$$T \rightarrow T * F$$
$$T \rightarrow F$$
$$F \rightarrow (E)$$
$$F \rightarrow a$$

is non-ambiguous:

Every string $w \in L(G)$ has
a unique derivation tree

Ambiguous Grammar

1. $S \rightarrow aSb \mid SS \mid e$

left derivation

$S \rightarrow aSb \rightarrow aaSbb \rightarrow aaeabb$

Left Derivation

$S \rightarrow SS \rightarrow aSbe \rightarrow aaSbb \rightarrow aabb$

2. $S \rightarrow iCtS \mid iCt SeS$

$C \rightarrow b$ $S \rightarrow a$ check for string "ibt ibt aea"

3. $S \rightarrow AB \mid aaB$

$A \rightarrow a \mid Aa$

$B \rightarrow b$ check for string aab

Inherent Ambiguity

Some context free languages
have only ambiguous grammars

Example: $L = \{a^n b^n c^m\} \cup \{a^n b^m c^m\}$

$$\begin{array}{lll} S \rightarrow S_1 \mid S_2 & S_1 \rightarrow S_1 c \mid A & S_2 \rightarrow a S_2 \mid B \\ A \rightarrow a A b \mid \lambda & & B \rightarrow b B c \mid \lambda \end{array}$$

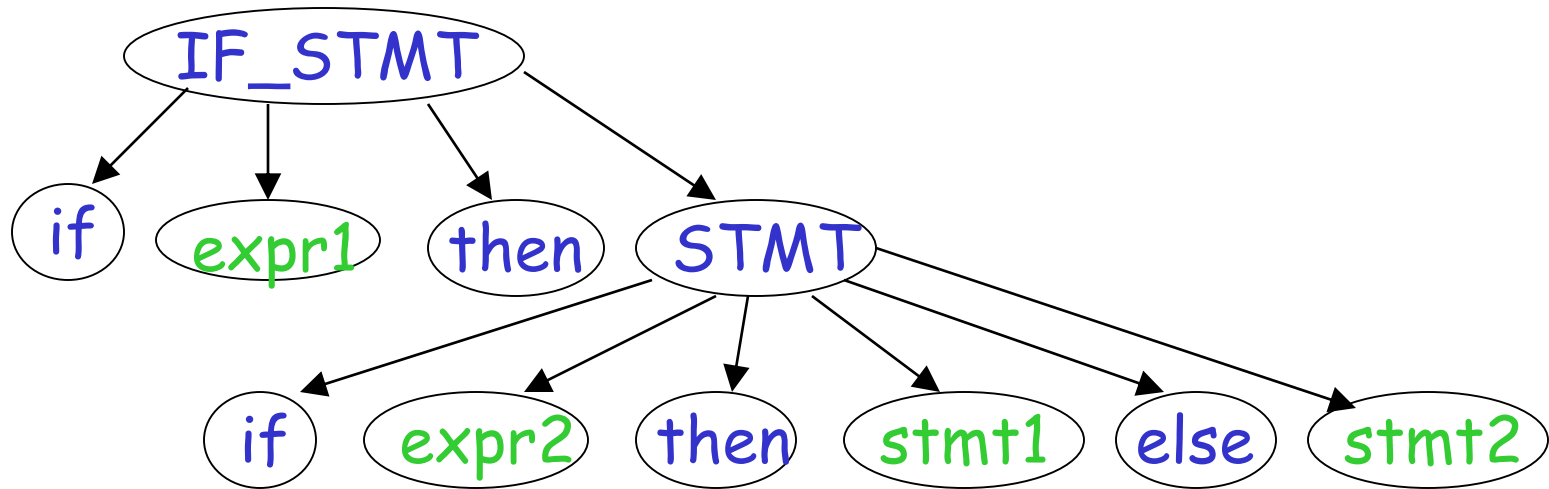
Another ambiguous grammar:

IF_STMT $\rightarrow$ if EXPR then STMT
 | if EXPR then STMT else IF_STMT

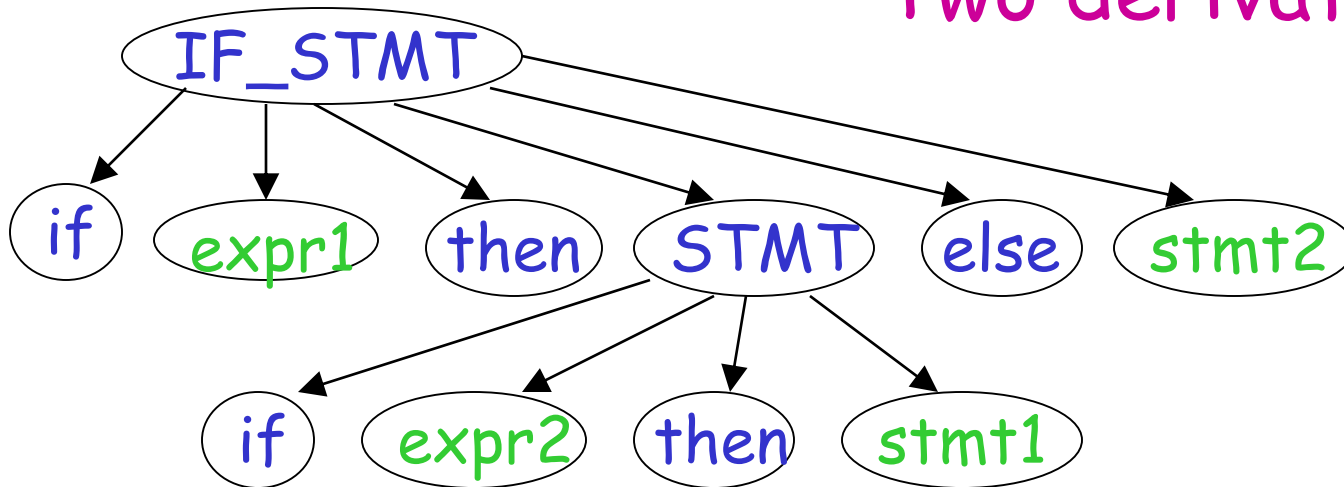
Variables Terminals

Very common piece of grammar
in programming languages

If expr1 then if expr2 then stmt1 else stmt2



Two derivation trees



An un-successful example:

$$L = \{a^n b^n c^m\} \cup \{a^n b^m c^m\}$$

$$n, m \geq 0$$

L is inherently ambiguous:

every grammar that generates this language is ambiguous

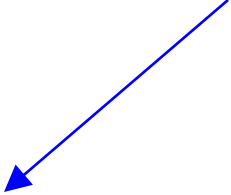
Nullable Variables

λ – production : $A \rightarrow \lambda$

Nullable Variable: $A \Rightarrow \dots \Rightarrow \lambda$

Example (ambiguous) grammar for L :

$$L = \{a^n b^n c^m\} \cup \{a^n b^m c^m\}$$


$$S \rightarrow S_1 \mid S_2$$


$$S_1 \rightarrow S_1 c \mid A$$

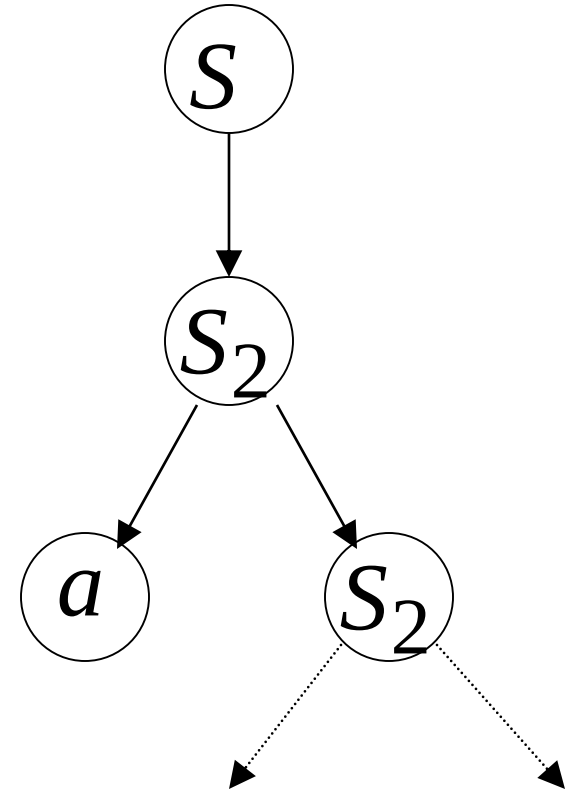
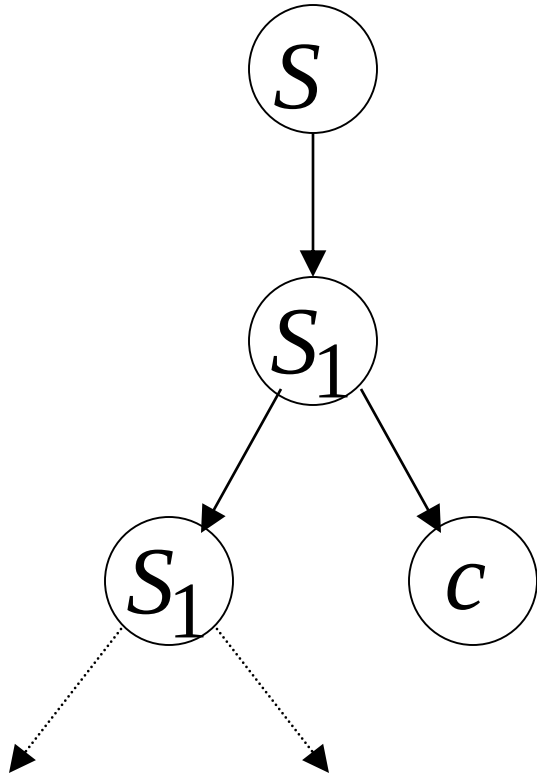
$$A \rightarrow aAb \mid \lambda$$


$$S_2 \rightarrow aS_2 \mid B$$

$$B \rightarrow bBc \mid \lambda$$

The string $a^n b^n c^n \in L$
has always two different derivation trees
(for any grammar)

For example



Simplifications of Context-Free Grammars

Removing All

Step 1: Remove Nullable Variables

Step 2: Remove Unit-Productions

Step 3: Remove Useless Variables

A Substitution Rule

$$S \rightarrow aB$$

$$A \rightarrow aaA$$

$$A \rightarrow abBc$$

$$B \rightarrow aA$$

$$B \rightarrow b$$

Substitute

$$B \rightarrow b$$

Equivalent
grammar

$$S \rightarrow aB \mid ab$$

$$A \rightarrow aaA$$

$$A \rightarrow abBc \mid abbc$$

$$B \rightarrow aA$$

A Substitution Rule

$$S \rightarrow aB \mid ab$$

$$A \rightarrow aaA$$

$$A \rightarrow abBc \mid abbc$$

$$B \rightarrow aA$$

Substitute

$$B \rightarrow aA$$

$$S \rightarrow \cancel{aB} \mid ab \mid aaA$$

$$A \rightarrow aaA$$

$$A \rightarrow \cancel{abBc} \mid abbc \mid abaAc$$

Equivalent
grammar

In general:

$$A \rightarrow xBz$$

$$B \rightarrow y_1$$

Substitute

$$B \rightarrow y_1$$

$$A \rightarrow xBz \mid xy_1z$$

equivalent
grammar

Removing Nullable Variables

Example Grammar:

$$S \rightarrow aMb$$

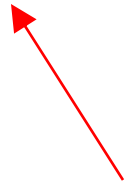
$$M \rightarrow aMb$$

$$M \rightarrow \lambda$$

Nullable variable



λ – production



Removing λ – productions

$$S \rightarrow aMb$$

$$M \rightarrow aMb$$

~~$$M \rightarrow \lambda$$~~

Substitute

$$M \rightarrow \lambda$$

$$S \rightarrow aMb \mid ab$$

$$M \rightarrow aMb \mid ab$$

After we remove all the λ – productions
all the nullable variables disappear
(except for the start variable)

Unit-Productions

Unit Production: $A \rightarrow B$

(a single variable in both sides)

Removing Unit Productions

Observation:

$$A \rightarrow A$$

Is removed immediately

Example Grammar:

$$S \rightarrow aA$$

$$A \rightarrow a$$

$$A \rightarrow B$$

$$B \rightarrow A$$

$$B \rightarrow bb$$

$$S \rightarrow aA$$

$$A \rightarrow a$$

~~$$A \rightarrow B$$~~

$$B \rightarrow A$$

$$B \rightarrow bb$$

Substitute

$$A \rightarrow B$$

$$S \rightarrow aA \mid aB$$

$$A \rightarrow a$$

$$B \rightarrow A \mid B$$

$$B \rightarrow bb$$

$$S \rightarrow aA \mid aB$$

$$A \rightarrow a$$

$$B \rightarrow A \mid \cancel{B}$$

$$B \rightarrow bb$$

Remove

$$B \rightarrow B$$

$$S \rightarrow aA \mid aB$$

$$A \rightarrow a$$

$$B \rightarrow A$$

$$B \rightarrow bb$$

$$S \rightarrow aA \mid aB$$

$$A \rightarrow a$$

~~$$B \rightarrow A$$~~

$$B \rightarrow bb$$

Substitute

$$B \rightarrow A$$

$$S \rightarrow aA \mid aB \mid aA$$

$$A \rightarrow a$$

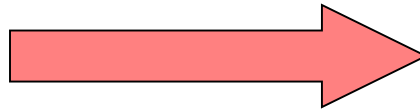
$$B \rightarrow bb$$

Remove repeated productions

$$S \rightarrow aA \mid aB \mid \cancel{aA}$$

$$A \rightarrow a$$

$$B \rightarrow bb$$



Final grammar

$$S \rightarrow aA \mid aB$$

$$A \rightarrow a$$

$$B \rightarrow bb$$

Useless Productions

$$S \rightarrow aSb$$

$$S \rightarrow \lambda$$

$$S \rightarrow A$$

$$A \rightarrow aA$$
 Useless Production

Some derivations never terminate...

$$S \Rightarrow A \Rightarrow aA \Rightarrow aaA \Rightarrow \dots \Rightarrow aa \dots aA \Rightarrow \dots$$

Another grammar:

$$S \rightarrow A$$

$$A \rightarrow aA$$

$$A \rightarrow \lambda$$

$$B \rightarrow bA$$

Useless Production

Not reachable from S

In general:

if

$$S \Rightarrow \dots \Rightarrow xAy \Rightarrow \dots \Rightarrow w$$

contains only
terminals



$$w \in L(G)$$

then variable A is useful

otherwise, variable A is useless

A production $A \rightarrow x$ is useless
if any of its variables is useless

$$S \rightarrow aSb$$

$$S \rightarrow \lambda$$

Productions

Variables

$$S \rightarrow A$$

useless

useless

$$A \rightarrow aA$$

useless

useless

$$B \rightarrow C$$

useless

useless

$$C \rightarrow D$$

useless

Removing Useless Productions

Example Grammar:

$$S \rightarrow aS \mid A \mid C$$

$$A \rightarrow a$$

$$B \rightarrow aa$$

$$C \rightarrow aCb$$

First: find all variables that can produce strings with only terminals

$$S \rightarrow aS \mid A \mid C$$

Round 1: $\{A, B\}$

$$A \rightarrow a$$

$$S \rightarrow A$$

$$B \rightarrow aa$$

$$C \rightarrow aCb$$

Round 2: $\{A, B, S\}$

Keep only the variables
that use variables : $\{A, B, S\}$

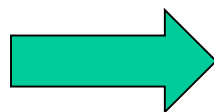
(the rest variables are useless)

$$S \rightarrow aS \mid A \mid \cancel{C}$$

$$A \rightarrow a$$

$$B \rightarrow aa$$

$$\cancel{C \rightarrow aCb}$$



$$S \rightarrow aS \mid A$$

$$A \rightarrow a$$

$$B \rightarrow aa$$

Remove useless productions

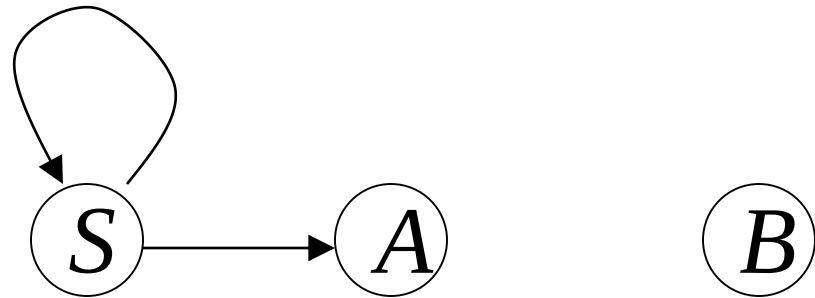
Second: Find all variables
reachable from S

Use a Dependency Graph

$S \rightarrow aS \mid A$

$A \rightarrow a$

$B \rightarrow aa$



unreachable

Note: nodes are variables

Keep only the variables
reachable from S

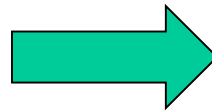
(the rest variables are useless)

Final Grammar

$$S \rightarrow aS \mid A$$

$$A \rightarrow a$$

~~$$B \rightarrow aa$$~~



$$S \rightarrow aS \mid A$$

$$A \rightarrow a$$

Remove useless productions

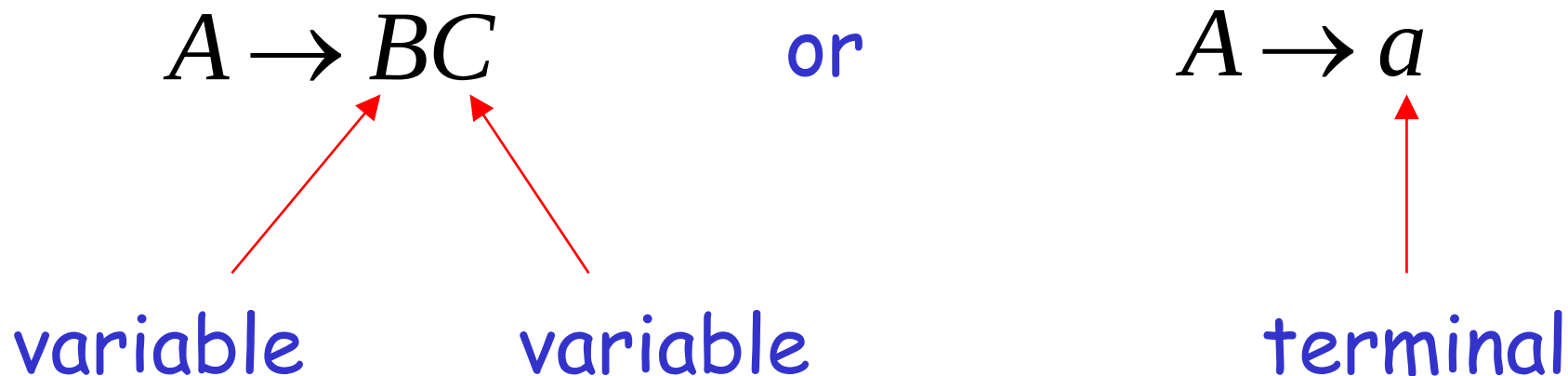
Normal Forms for Context-free Grammars

Chomsky Normal Form

Each productions has form:

Nonterminal $\rightarrow$ string of only two Nonterminal

Nonterminal $\rightarrow$ one terminal



CNF STEPS

Steps to convert a given CFG to Chomsky Normal Form:

- Step 1: If the Start Symbol S occurs on some right side, create a new Start Symbol S' and a new Production $S' \rightarrow S$.
- Step 2: Remove Null Productions. (Using the Null Production Removal discussed in previous Lecture)
- Step 3: Remove Unit Productions. (Using the Unit Production Removal discussed in previous Lecture)
- Step 4: Replace each Production $A \rightarrow B_1 \dots B_n$ where $n > 2$, with $A \rightarrow B_1 C$ where $C \rightarrow B_2 \dots B_n$. Repeat this step for all Productions having two or more Symbols on the right side.
- Step 5: If the right side of any Production is in the form $A \rightarrow aB$ where 'a' is a terminal and A and B are non-terminals, then the Production is replaced by $A \rightarrow XB$ and $X \rightarrow a$. Repeat this step for every Production which is of the form $A \rightarrow aB$.



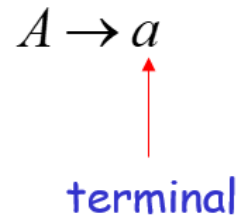
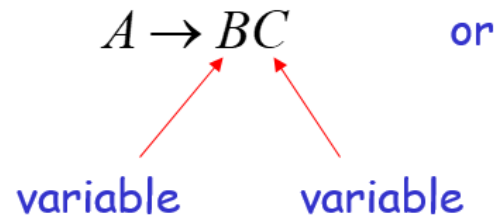
Example

$$S \rightarrow ASA \mid aB$$

$$A \rightarrow B \mid S$$

$$B \rightarrow b \mid \lambda$$

Not in CNF,
because it
not follow
the rules of
CNF

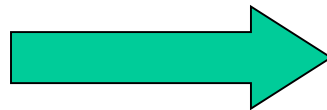


new symbol $\bar{S}^1$ and new production

$S \rightarrow ASA \mid aB$

$A \rightarrow B \mid S$

$B \rightarrow b \mid \lambda$



$\bar{S}^1 \rightarrow S$

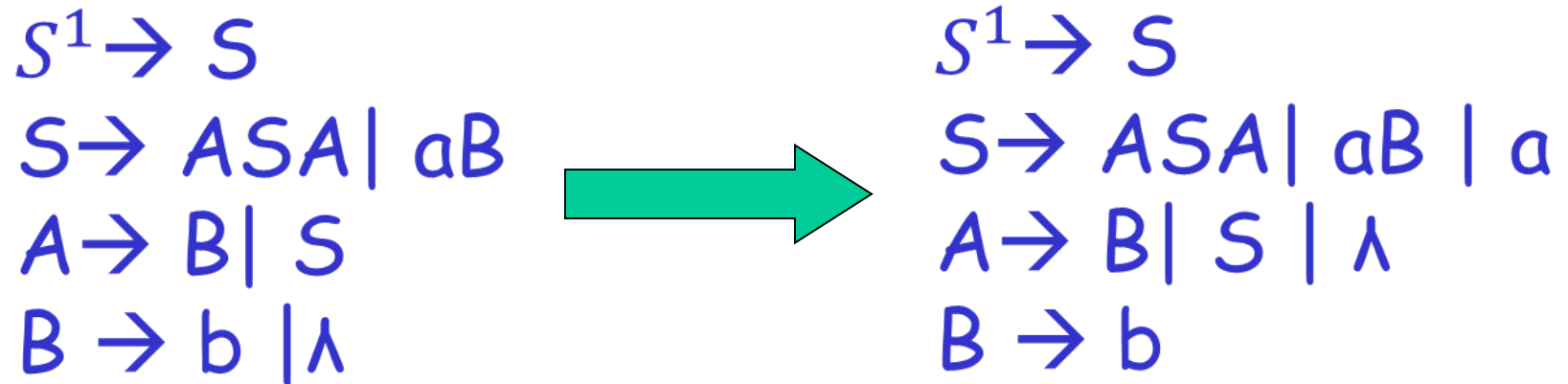
$S \rightarrow ASA \mid aB$

$A \rightarrow B \mid S$

$B \rightarrow b \mid \lambda$

Step # 2 Remove Null Productions

Removing $B \rightarrow \lambda$



Step # 2 Remove Null Productions

Removing $A \rightarrow \lambda$

$$\begin{array}{l} S^1 \rightarrow S \\ S \rightarrow ASA \mid aB \mid \\ A \rightarrow B \mid S \mid \lambda \\ B \rightarrow b \end{array} \quad \longrightarrow \quad \begin{array}{l} S^1 \rightarrow S \\ S \rightarrow ASA \mid aB \mid a \mid AS \mid SA \mid S \\ A \rightarrow B \mid S \\ B \rightarrow b \end{array}$$

Step # 3 Remove Unit Productions

$$S^1 \rightarrow S$$

$$S \rightarrow ASA \mid aB \mid a \mid AS \mid SA \mid S$$

$$A \rightarrow B \mid S$$

$$B \rightarrow b$$

Remove $S \rightarrow S$

$$S^1 \rightarrow S$$

$$S \rightarrow ASA \mid aB \mid a \mid AS \mid SA$$

$$A \rightarrow B \mid S$$

$$B \rightarrow b$$

Step # 3 Remove Unit Productions

$$S^1 \rightarrow S$$

$$S \rightarrow ASA \mid aB \mid a \mid AS \mid SA \mid S$$

$$A \rightarrow B \mid S$$

$$B \rightarrow b$$

Remove $S^1 \rightarrow S$

$$S^1 \rightarrow ASA \mid aB \mid a \mid AS \mid SA$$

$$S \rightarrow ASA \mid aB \mid a \mid AS \mid SA$$

$$A \rightarrow B \mid S$$

$$B \rightarrow b$$

Step # 3 Remove Unit Productions

$S^1 \rightarrow ASA \mid aB \mid a \mid AS \mid SA$

$S \rightarrow ASA \mid aB \mid a \mid AS \mid SA$

$A \rightarrow B \mid S$

$B \rightarrow b$

Remove $A \rightarrow B$

$S^1 \rightarrow ASA \mid aB \mid a \mid AS \mid SA$

$S \rightarrow ASA \mid aB \mid a \mid AS \mid SA$

$A \rightarrow b \mid S$

$B \rightarrow b$

Step # 3 Remove Unit Productions

$S^1 \rightarrow ASA \mid aB \mid a \mid AS \mid SA$

$S \rightarrow ASA \mid aB \mid a \mid AS \mid SA$

$A \rightarrow b \mid S$

$B \rightarrow b$

Remove $A \rightarrow S$

$S^1 \rightarrow ASA \mid aB \mid a \mid AS \mid SA$

$S \rightarrow ASA \mid aB \mid a \mid AS \mid SA$

$A \rightarrow b \mid ASA \mid aB \mid a \mid AS \mid SA$

$B \rightarrow b$

Step # 4 Remove More than two variables on RHS

$S^1 \rightarrow ASA \mid aB \mid a \mid AS \mid SA$

$S \rightarrow ASA \mid aB \mid a \mid AS \mid SA$

$A \rightarrow b \mid ASA \mid aB \mid a \mid AS \mid SA$

$B \rightarrow b$

Remove $S^1 \rightarrow ASA$, $S \rightarrow ASA$ and $A \rightarrow ASA$

$S^1 \rightarrow AX \mid aB \mid a \mid AS \mid SA$

$S \rightarrow AX \mid aB \mid a \mid AS \mid SA$

$A \rightarrow b \mid AX \mid aB \mid a \mid AS \mid SA$

$B \rightarrow b$

$X \rightarrow SA$

Step # 5

$S^1 \rightarrow AX \mid aB \mid a \mid AS \mid SA$

$S \rightarrow AX \mid aB \mid a \mid AS \mid SA$

$A \rightarrow b \mid AX \mid aB \mid a \mid AS \mid SA$

$B \rightarrow b$

$X \rightarrow SA$

Remove $S^1 \rightarrow aB$, $S \rightarrow aB$ and $A \rightarrow aB$

$S^1 \rightarrow AX \mid YB \mid a \mid AS \mid SA$

$S \rightarrow AX \mid YB \mid a \mid AS \mid SA$

$A \rightarrow b \mid AX \mid YB \mid a \mid AS \mid SA$

$B \rightarrow b$

$X \rightarrow SA$

$Y \rightarrow a$

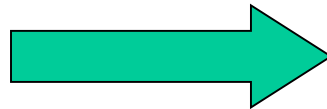
Example # 2

Step 1: We will create a new production $S1 \rightarrow S$, as the start symbol S appears on the RHS.

$S \rightarrow a \mid aA \mid B$

$A \rightarrow aBB \mid \epsilon$

$B \rightarrow Aa \mid b$



$S1 \rightarrow S$

$S \rightarrow a \mid aA \mid B$

$A \rightarrow aBB \mid \epsilon$

$B \rightarrow Aa \mid b$

Step 2: As grammar G_1 contains $A \rightarrow \varepsilon$ null production, its removal from the grammar yields:

$S_1 \rightarrow S$

$S \rightarrow a \mid aA \mid B$

$A \rightarrow aBB \mid \varepsilon$

$B \rightarrow Aa \mid b$



$S_1 \rightarrow S$

$S \rightarrow a \mid aA \mid B$

$A \rightarrow aBB$

$B \rightarrow Aa \mid b \mid a$

Now, as grammar G_1 contains Unit production $S \rightarrow B$, its removal yield:

$S_1 \rightarrow S$

$S \rightarrow a \mid aA \mid B$

$A \rightarrow aBB \mid \varepsilon$

$B \rightarrow Aa \mid b$



$S_1 \rightarrow S$

$S \rightarrow a \mid aA \mid Aa \mid b$

$A \rightarrow aBB$

$B \rightarrow Aa \mid b \mid a$

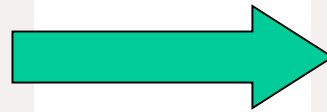
Also remove the unit production $S1 \rightarrow S$, its removal from the grammar yields:

$S1 \rightarrow S$

$S \rightarrow a \mid aA \mid Aa \mid b$

$A \rightarrow aBB$

$B \rightarrow Aa \mid b \mid a$



$S0 \rightarrow a \mid aA \mid Aa \mid b$

$S \rightarrow a \mid aA \mid Aa \mid b$

$A \rightarrow aBB$

$B \rightarrow Aa \mid b \mid a$

Step 3: In the production rule $S0 \rightarrow aA \mid Aa$, $S \rightarrow aA \mid Aa$, $A \rightarrow aBB$ and $B \rightarrow Aa$, terminal a exists on RHS with non-terminals. So we will replace terminal a with X :

```
S0 → a | aA | Aa | b
S → a | aA | Aa | b
A → aBB
B → Aa | b | a
```



```
S0 → a | XA | AX | b
S → a | XA | AX | b
A → XBB
B → AX | b | a
X → a
```


Step 4: In the production rule $A \rightarrow XBB$, RHS has more than two symbols, removing it from grammar yield:

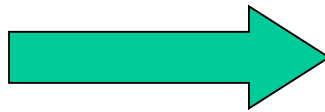
$S_0 \rightarrow a \mid XA \mid AX \mid b$

$S \rightarrow a \mid XA \mid AX \mid b$

$A \rightarrow XBB$

$B \rightarrow AX \mid b \mid a$

$X \rightarrow a$



$S_0 \rightarrow a \mid XA \mid AX \mid b$

$S \rightarrow a \mid XA \mid AX \mid b$

$A \rightarrow RB$

$B \rightarrow AX \mid b \mid a$

$X \rightarrow a$

$R \rightarrow XB$